



โครงสร้างข้อมูลต้นไม้ (Tree Data Structure)

โดย ผศ.ดร.เชาวลิต ชั่นคำ

ภาควิชาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยรามคำแหง

- 
- เพื่อให้ผู้ศึกษา
 - 1. มีความรู้เกี่ยวกับแนวคิด ความหมาย ประเภทของต้นไม้และต้นไม้ค้นหาทวิภาค
 - 2. มีความรู้เกี่ยวกับนิยาม คุณลักษณะ ข้อจำกัดของต้นไม้ค้นหาทวิภาค
 - 3. มีความรู้เกี่ยวกับการท่องต้นไม้ค้นหาทวิภาคแบบพรีออร์เดอร์ อินออร์เดอร์ และโพสต์ออร์เดอร์
 - 4. มีความรู้เกี่ยวกับการแทรกข้อมูลในต้นไม้ค้นหาทวิภาค
 - 5. มีความรู้เกี่ยวกับการค้นหาข้อมูลในต้นไม้ค้นหาทวิภาค
 - 7. มีความรู้เกี่ยวกับการลบข้อมูลในต้นไม้ค้นหาทวิภาค
 - 7. มีความรู้เกี่ยวกับการนำต้นไม้ค้นหาทวิภาคไปประยุกต์ใช้งาน

- 
- การเข้าถึงข้อมูลในโครงสร้างข้อมูลที่ได้นำเสนอไปแล้วนั้น ต้องเข้าถึงตามลำดับ เพราะเป็นข้อมูลแบบต่อเนื่อง จึงทำให้การใช้งานโครงสร้างข้อมูลอาจยังไม่ค่อยมีประสิทธิภาพมากนัก เนื่องจากข้อจำกัดการเข้าถึงและดำเนินการตามลำดับจากข้อมูลที่อยู่อย่างต่อเนื่องนั่นเอง อีกทั้ง ความจำเป็นในปัญหาบางอย่างที่โครงสร้างข้อมูลแบบต่อเนื่องไม่สามารถตอบได้ ดังนั้น โครงสร้างข้อมูลแบบไม่ต่อเนื่องจึงจำเป็น เพื่อแก้ปัญหาการเข้าถึงและการจัดเก็บ โดยพิจารณาจากการเชื่อมโยงระหว่างรายการต่อรายการในอีกแนวคิดหนึ่ง โดยเฉพาะความสัมพันธ์แบบลำดับชั้น (**Hierarchical**) พิจารณาจากบนลงล่าง ลักษณะการเข้าถึงข้อมูลไม่จำเป็นต้องเข้าถึงตามลำดับ ขึ้นอยู่กับลักษณะของอัลกอริทึมการท่องที่แตกต่างกัน โครงสร้างต้นไม้ (**Tree Structure**) จึงเป็นคำตอบ เพราะสนับสนุนการแทรกและการค้นหาข้อมูลได้อย่างรวดเร็ว การเข้าถึงแบบลำดับชั้น สามารถท่องได้ในหลากหลายรูปแบบ เนื้อหาในบทนี้ นำเสนอเกี่ยวกับความหมายของต้นไม้ ประเภทของต้นไม้ ต้นไม้แบบต้นไม้ทวิภาคและต้นไม้ค้นหาทวิภาค การท่อง เพิ่มโนด การลบโนด การค้นหา การลบข้อมูลในต้นไม้ค้นหาทวิภาค นอกจากนั้นนำเสนอแนวทางประยุกต์ใช้ได้อีกด้วย

ความหมายและแนวคิด

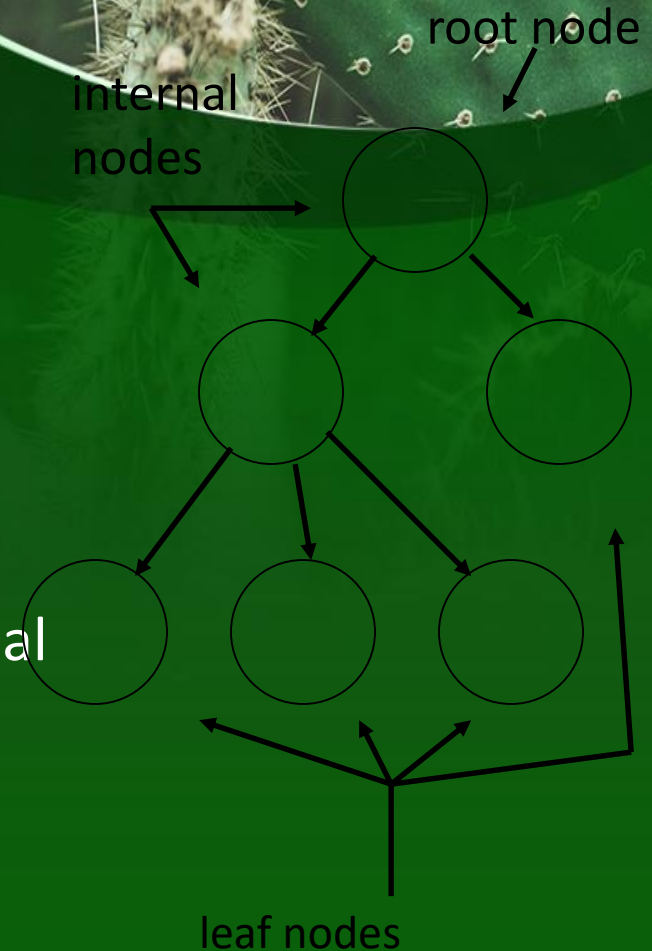
- ทรีเป็นข้อมูลเชิงนามธรรม (**Abstract Data Type**)
- ทรีเป็นโครงสร้างข้อมูลที่ไม่ต่อเนื่อง (**non-linear data structure**) ที่ประกอบด้วยเซตของโนด โดยที่อนุญาตให้ในแต่ละทรีนั้นสามารถเป็นทรีว่างได้ (**null or empty tree**)
- ใช้ความสัมพันธ์แบบลำดับชั้นในการเชื่อมโยงข้อมูลแต่ละอีลิเมนต์
- ข้อมูลแต่ละอีลิเมนต์เรียกว่าโนด
- โหนดแม่ของทุกโนดเรียกว่า ราก (**root**) ส่วนอื่นๆ ที่เชื่อมโยงต่อจากนั้น เรียกว่า ทรีย่อย (**sub-tree**)

ความรู้เบื้องต้นเกี่ยวกับต้นไม้

- ต้นไม้เป็นโครงสร้างข้อมูลเชิงนามธรรม (**Abstract Data Type**) ที่ไม่ต่อเนื่อง (**Non-Linear Data Structure**) ประกอบด้วยเซตของโนด โดยอนุญาตให้ในแต่ละต้นไม้นั้นสามารถเป็นต้นไม้ว่างได้ (**Null or Empty Tree**) ใช้ความสัมพันธ์แบบลำดับชั้นในการเชื่อมโยงข้อมูลแต่ละรายการ ข้อมูลแต่ละรายการเรียกว่าโนด โหนดแม่ของทุกโนดเรียกว่า ราก (**Root**) ส่วนอื่นๆ ที่เชื่อมโยงต่อจากนั้น เรียกว่า ต้นไม้ย่อย (**Sub-Tree**) ลักษณะของต้นไม้คล้ายต้นไม้หัวกลับ

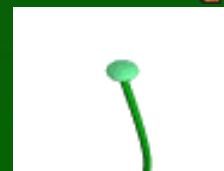
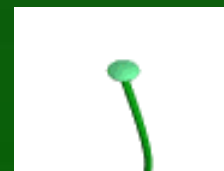
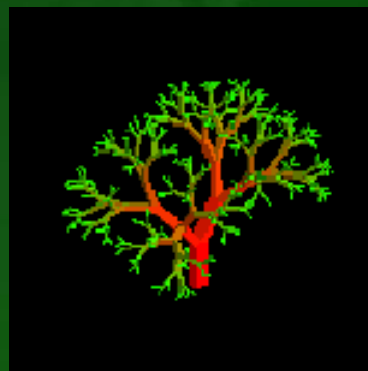
Definitions

- ตรีเป็น abstract data type ที่
 - จะมีโนดหนึ่ง เป็นราก one entry point, the **root**
 - แต่ละโนด ไม่เป็น **leaf** ก็เป็น **internal node**
 - แต่ละ internal node มี 1 **children** หรือมากกว่า โนด จะสามารถเข้าถึงได้โดยตรงจาก internal node.
 - กล่าวได้ว่า internal node คือ **parent** ของ child โนด

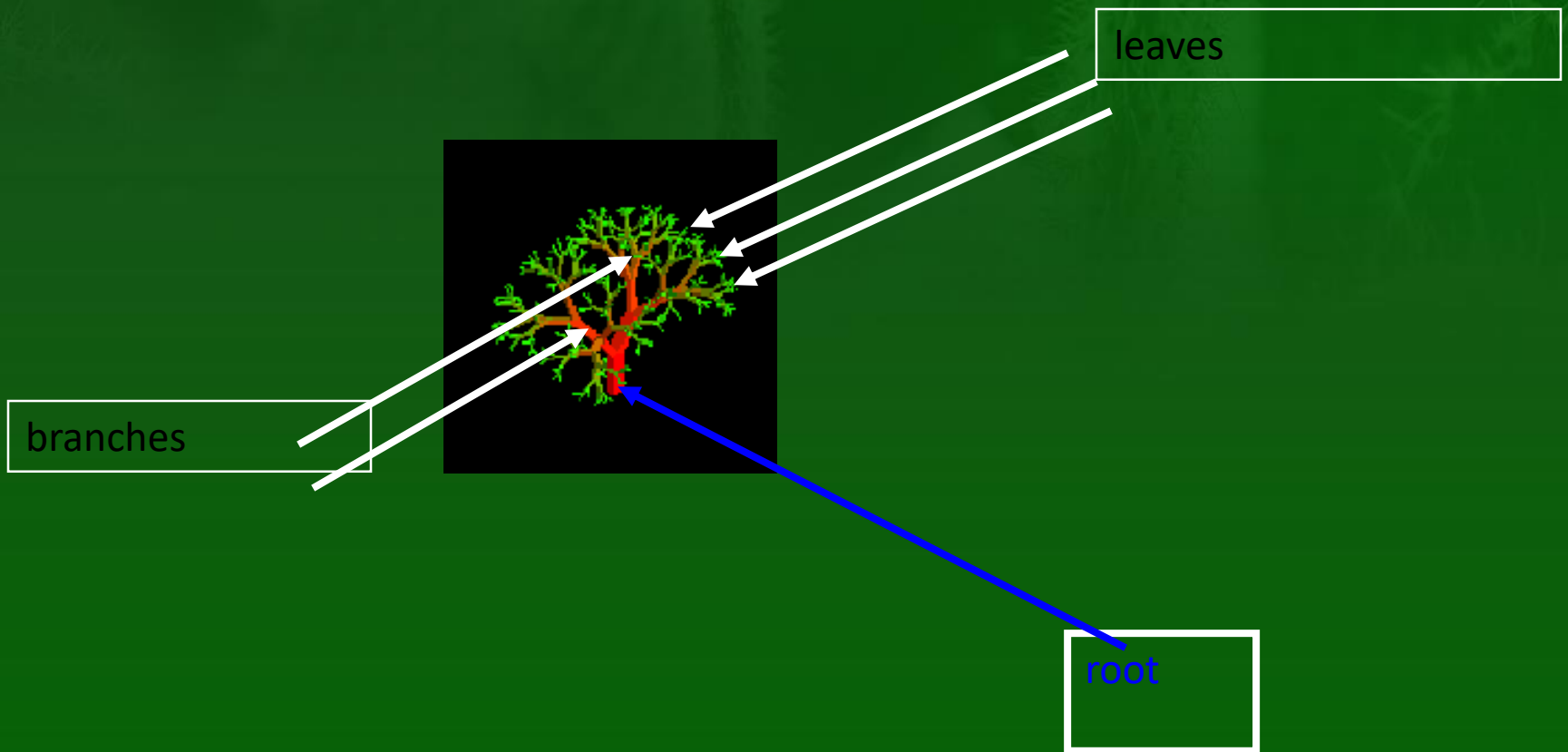




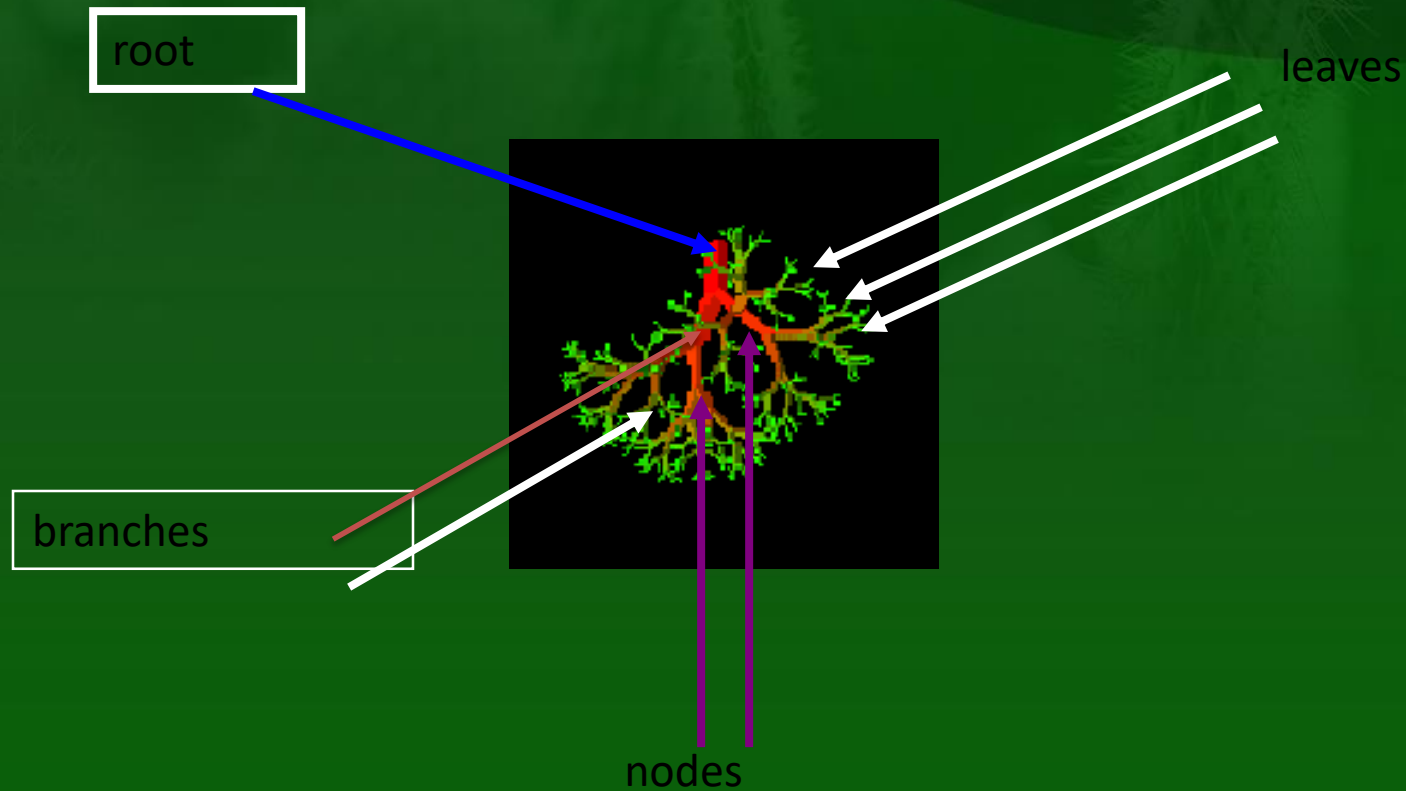
rees
ตัวอย่างจากที่มีอยู่แล้ว



Nature Lover's View Of A Tree



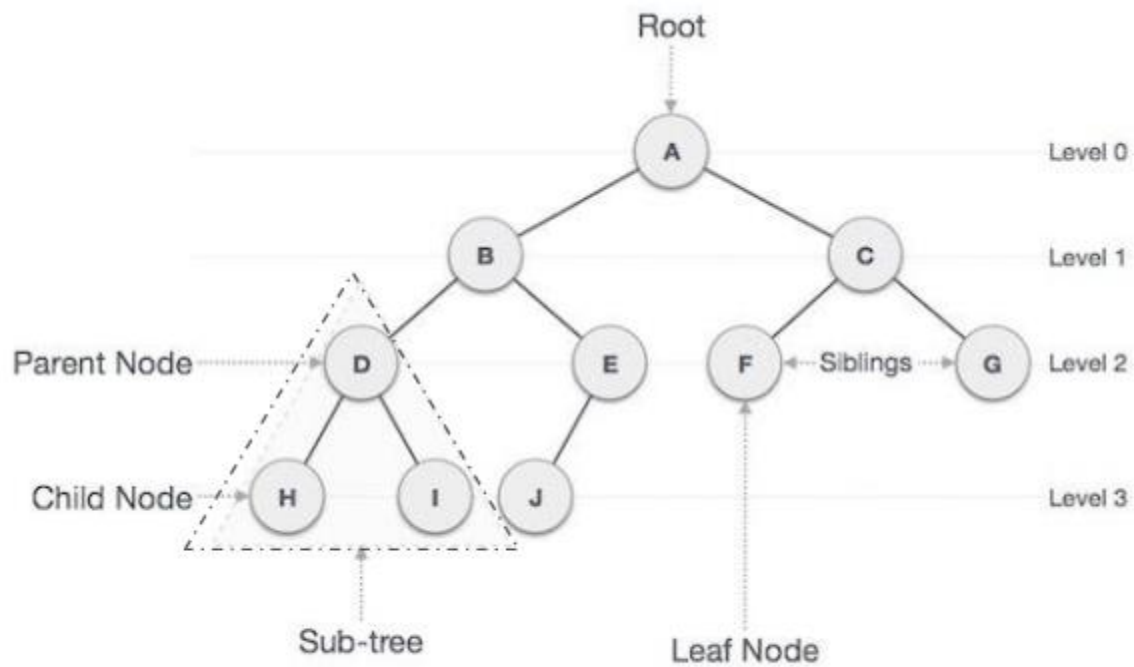
Computer Scientist's View



- 
- **นิยามที่ 7.1** โครงสร้างต้นไม้ (Tree Structure) คือ โครงสร้างข้อมูลไม่ต่อเนื่อง ภายในประกอบด้วยเซตของอีลิเมนต์ (Element) เรียกว่า โหนด (Node) เชื่อมโยงกันแบบลำดับชั้น โดยมีโหนดแรกสุดเป็น ราก (Root) เชื่อมโยงโครงสร้างต้นไม้ย่อย (Sub-Tree)

คำศัพท์เกี่ยวกับทรี

- **Root** – โหนดบนสุดของต้นไม้
- **Parent** – โหนดแม่ที่โหนดลูกเชื่อมต่ออยู่
- **Siblings** – โหนดข้างเคียงที่มีโหนดแม่เดียวกัน
- **Descendant** – โหนดที่สามารถเข้าถึงได้จากการท่องเที่ยวทางโหนดแม่สู่โหนดลูก
- **Ancestor** – โหนดที่สามารถเข้าถึงได้จากการท่องเที่ยวทางโหนดลูกเข้าหาโหนดแม่
- **Leaf** – โหนดปลายที่ไม่โหนดลูกเชื่อมต่อ
- **Internal Node** – โหนดที่มีลูกอย่างน้อยหนึ่งโหนด
- **External Node** – โหนดที่ไม่มีลูก
- **Degree** – จำนวนของต้นไม้ย่อยของโหนด
- **Edge** – การเชื่อมต่อระหว่างโหนดหนึ่งไปยังอีกโหนดอื่น
- **Path** – ลำดับของโหนดและเส้นเชื่อมของโหนดด้วยดีเซนแดนท์
- **Level** – ระดับของโหนดที่กำหนดโดยหนึ่งบวกกับจำนวนการเชื่อมต่อระหว่างโหนดและโหนดราก กำหนดโดย $1 + \text{จำนวนของเส้นเชื่อมต่อระหว่างโหนดกับราก}$
- **Height** – ความสูงของโหนด คือ ความยาวที่สุดจากโหนดสู่โหนดปลาย



ประเภทของโครงสร้างต้นไม้



ประเภทของทรี

- Binary trees
- Self-balancing binary search trees
- B-trees
- Tries
- Binary space partitioning (BSP) trees
- Non-binary trees
- Spatial data partitioning trees
- Other trees

ประเภทของทรี

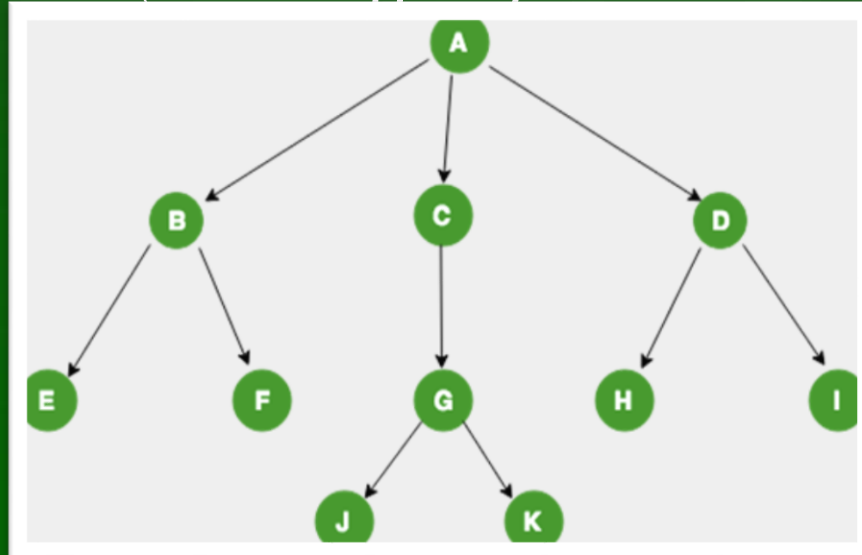
- Binary trees
 - Binary search tree (BST)
 - Cartesian tree
 - MVP Tree
 - Top tree
 - T-tree
 - Left-child right-sibling binary tree
- Self-balancing binary search trees
 - AA tree
 - AVL tree
 - LLRB tree
 - Red-black tree
 - Scapegoat tree
 - Splay tree
 - Treap
- B-trees
 - B+ tree
 - B*-tree
 - B^x-tree
 - **UB-tree**
 - 2–3 tree
 - 2–3–4 tree
 - (a,b)-tree
 - Dancing tree
 - HTree

ประเภทของทรี

- Tries
 - Suffix tree
 - Radix tree
 - Hash tree
 - Ternary search tree
 - X-fast trie
 - Y-fast trie
- Binary space partitioning (BSP) trees
 - Quadtree
 - Octree
 - k -d tree
 - Implicit k -d tree
 - VP tree
- Non-binary trees
 - Exponential tree
 - Fusion tree
 - Interval tree
 - PQ tree
 - Range tree
 - SPQR tree
 - Van Emde Boas tree
- Spatial data partitioning trees
 - R-tree
 - R+ tree
 - R* tree
 - X-tree
 - M-tree
 - Segment tree
 - Hilbert R-tree
 - Priority R-tree
- Other trees
 - Heap
 - Hash calendar
 - Merkle tree
 - Finger tree
 - Order statistic tree
 - Metric tree
 - Cover tree
 - BK-tree
 - Doubly chained tree
 - iDistance
 - Link/cut tree
 - Fenwick tree
 - Log-structured merge-tree

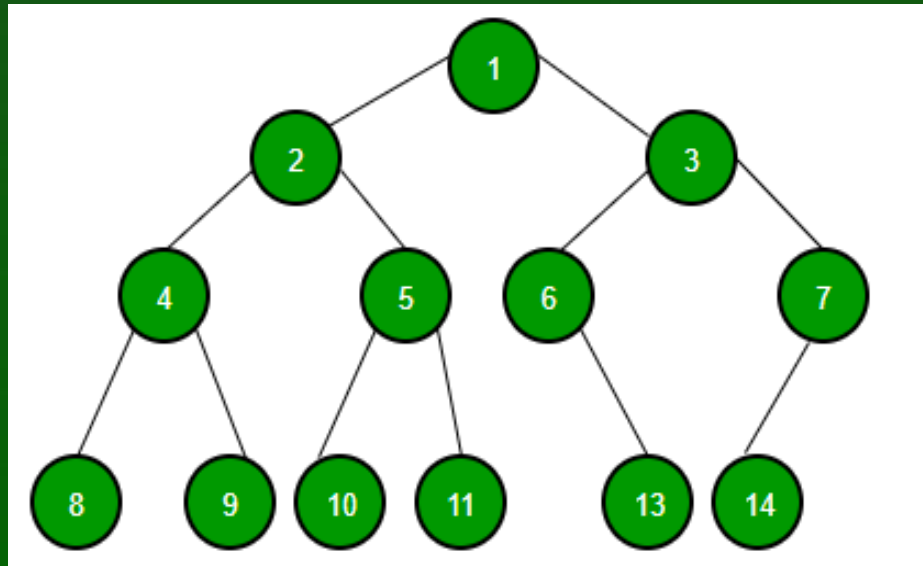
ต้นไม้ทั่วไป (General Tree)

- ต้นไม้ทั่วไป คือ ต้นไม้ที่แต่ละโหนดสามารถมีโหนดลูกเป็นศูนย์หรือหลายโหนดก็ได้ ที่สำคัญจะต้องไม่ว่างเปล่า ต้นไม้ทั่วไปไม่มีข้อจำกัดเกี่ยวกับระดับของโหนด ซึ่งโหนดบนสุดคือโหนดราก และสามารถมีต้นไม้ย่อยจำนวนมากได้ การนำเสนอข้อมูลในโหนดต่างๆ ที่บรรจุอยู่ภายในไม่จำเป็นต้องเรียงลำดับ แต่ละโหนดจะมีเส้นเข้า (In-Degree) เพียง 1 อาจมีเส้นออก (Out-Degree) ตั้งแต่ 1-n แสดงรูปภาพของต้นไม้ทั่วไปดังนี้



ต้นไม้ทวิภาค (Binary Tree)

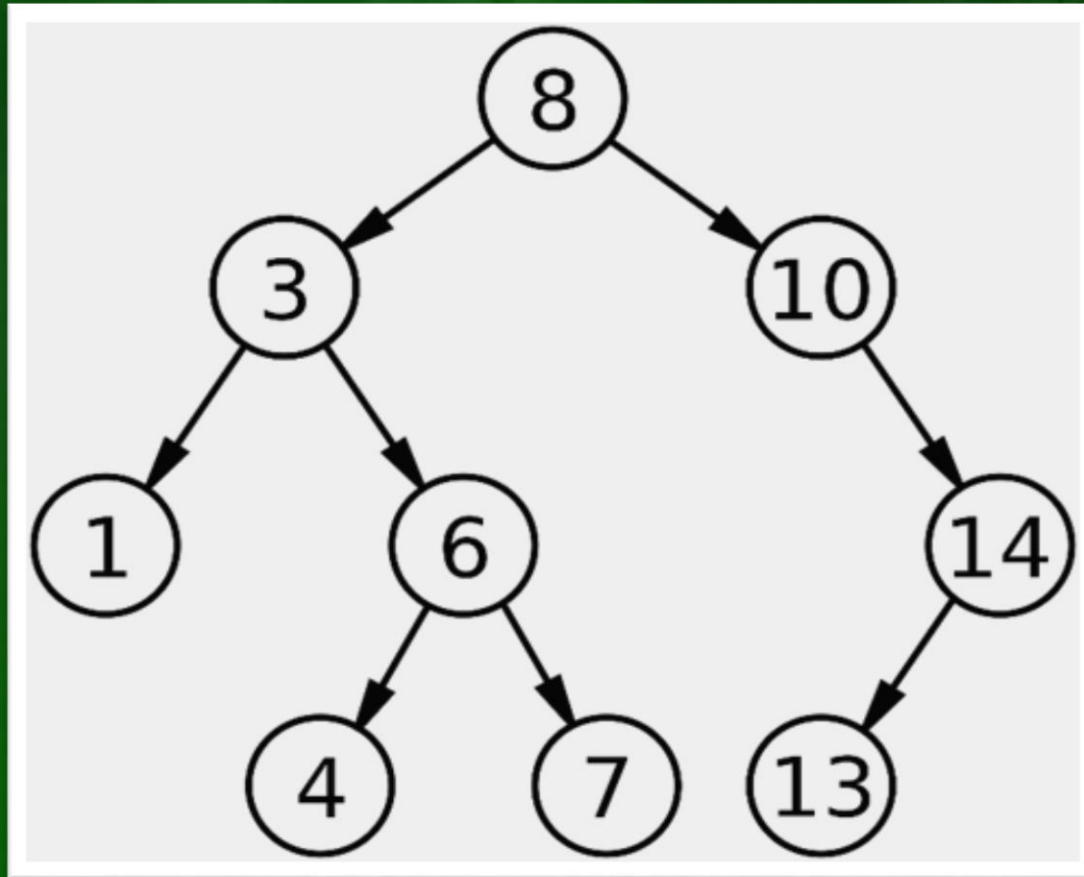
- ต้นไม้ทวิภาคคือต้นไม้ที่มีองค์ประกอบประกอบไม่เกิน 2 ตั้งชื่อลูกเหล่านั้นว่าลูกทางซ้าย (**Left Node**) และขวา (**Right Node**) ซึ่งหมายถึง การมีเส้นเชื่อมออกจากโหนดมีได้ไม่เกิน 2 เส้น และสามารถมีต้นไม้ย่อยได้ไม่เกินสองนั่นเอง แสดงตัวอย่างภาพของต้นไม้ทวิภาคได้ ดังนี้





ต้นไม้ค้นหาทวิภาค (Binary Search Tree--BST)

- ต้นไม้ค้นหาทวิภาคเป็นต้นไม้ทวิภาค ที่ช่วยให้สามารถรักษารายการเรียงลำดับเอาไว้เพื่อสามารถค้นหาได้รวดเร็วด้วยความซับซ้อน $O(\log n)$ เสมอหากต้นไม้ นั้นยังคงความสมดุล คุณสมบัติที่แยกต้นไม้ค้นหาทวิภาคออกจากต้นไม้ทวิภาค คือ โหนดทั้งหมดของต้นไม้ย่อยด้านซ้าย มีค่าน้อยกว่าโนดราก โหนดทั้งหมดของต้นไม้ย่อยด้านขวา มีมากกว่าโนดราก ต้นไม้แบบนี้นิยมใช้งานมากกว่าต้นไม้ทวิภาคธรรมดา โดยเฉพาะนำมาเก็บข้อมูลที่เป็นตัวเลข เพราะสามารถรักษาลำดับการจัดเรียงได้เสมอ เนื้อหาในบทนี้ จะนำเสนออัลกอริทึมสำหรับการจัดการต้นไม้ประเภทนี้เป็นหลัก แสดงตัวอย่างภาพของต้นไม้ประเภทนี้ ดังนี้

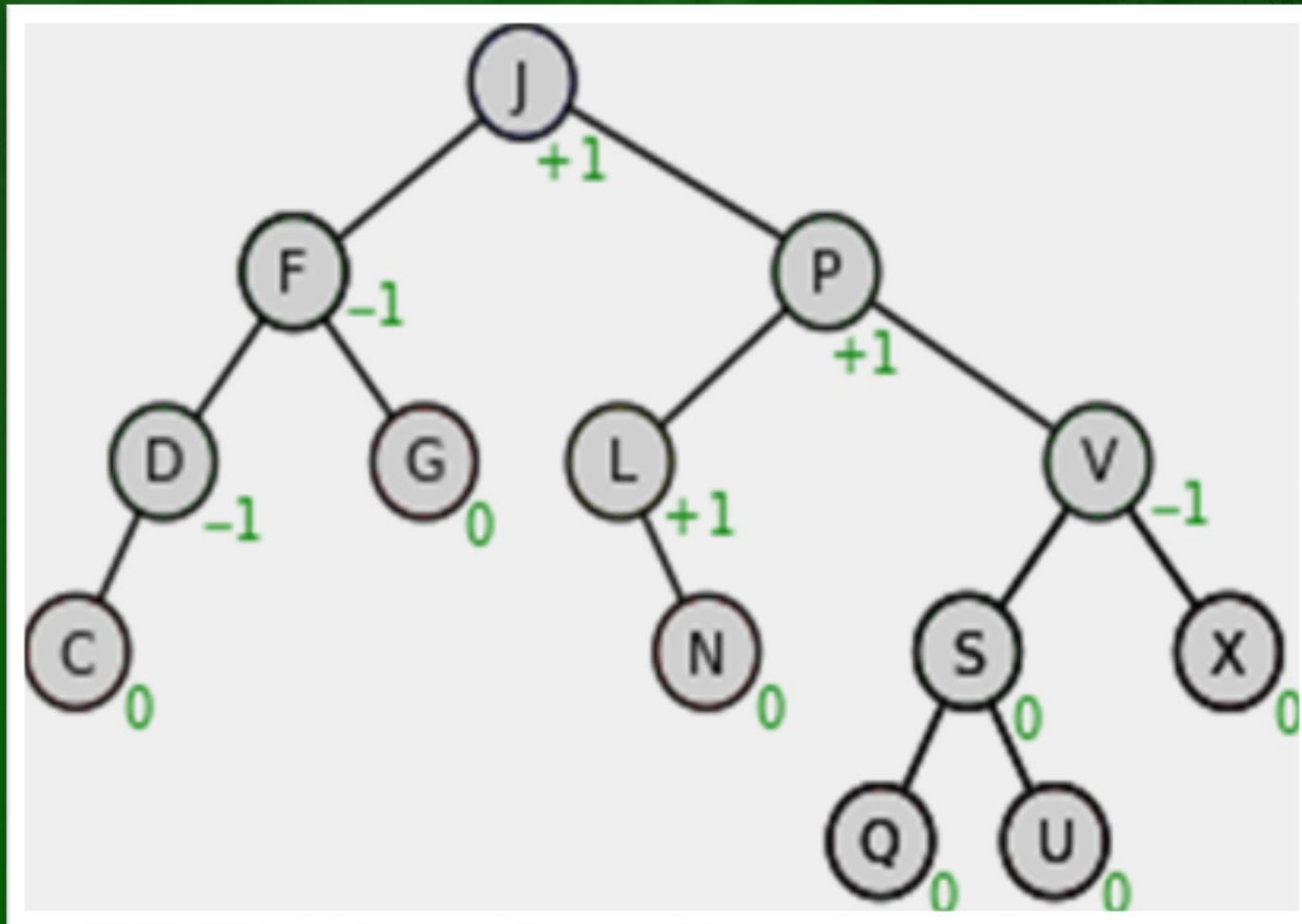


ต้นไม้สมดุล (Balancing Trees)

- ต้นไม้สมดุล เป็นแนวคิดของการนำเอาต้นไม้ค้นหาทวิภาคไปใช้งาน เพื่อรักษาประสิทธิภาพของการค้นหา ด้วยการทำให้ต้นไม้มีความสมดุลมากที่สุดเท่าที่สามารถทำได้ ซึ่งจะมีตัวดำเนินการสร้างการสมดุลหลังจากการเพิ่มหรือลบโนด โครงสร้างต้นไม้สมดุลที่มีชื่อเสียงและจะนำเสนอในบทต่อไป ได้แก่ ต้นไม้เอวีแอล และ ต้นไม้แดง-ดำ

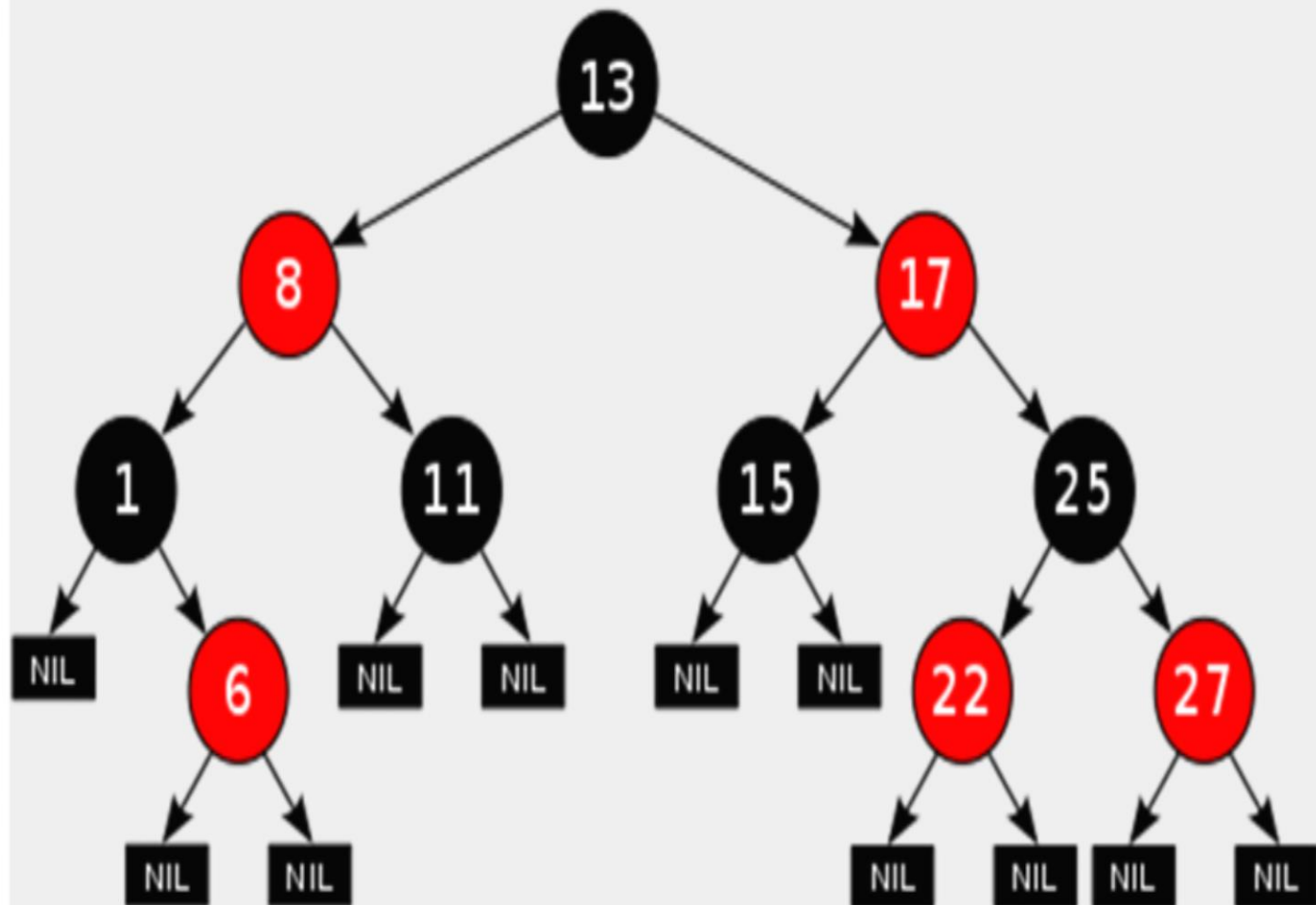
ต้นไม้เอวีแอล (AVL Tree)

- ต้นไม้เอวีแอลเป็นโครงสร้างต้นไม้ที่ปรับสมดุลในตัวเองได้ ตั้งชื่อตามผู้คิดค้นขึ้น คือ **Adelson-Velshi** และ **Landis** ในระดับความสูงของต้นไม้ จะมีการปรับค่าโดยให้มีความแตกต่างกันอยู่หนึ่งระดับเสมอ กำหนดปัจจัยสมดุล คือ **1**, **0** และ **-1** เมื่อมีการเพิ่มโหนดใหม่ หรือลบจะต้องปรับปัจจัยเหล่านี้ด้วยการหมุน และให้ค่าปัจจัยในเงื่อนไขที่ถูกต้อง จึงจะได้ต้นไม้ที่สมดุลเสมอ มีการใช้กันอย่างแพร่หลายสำหรับการดำเนินการค้นหา แสดงตัวอย่างภาพดังนี้



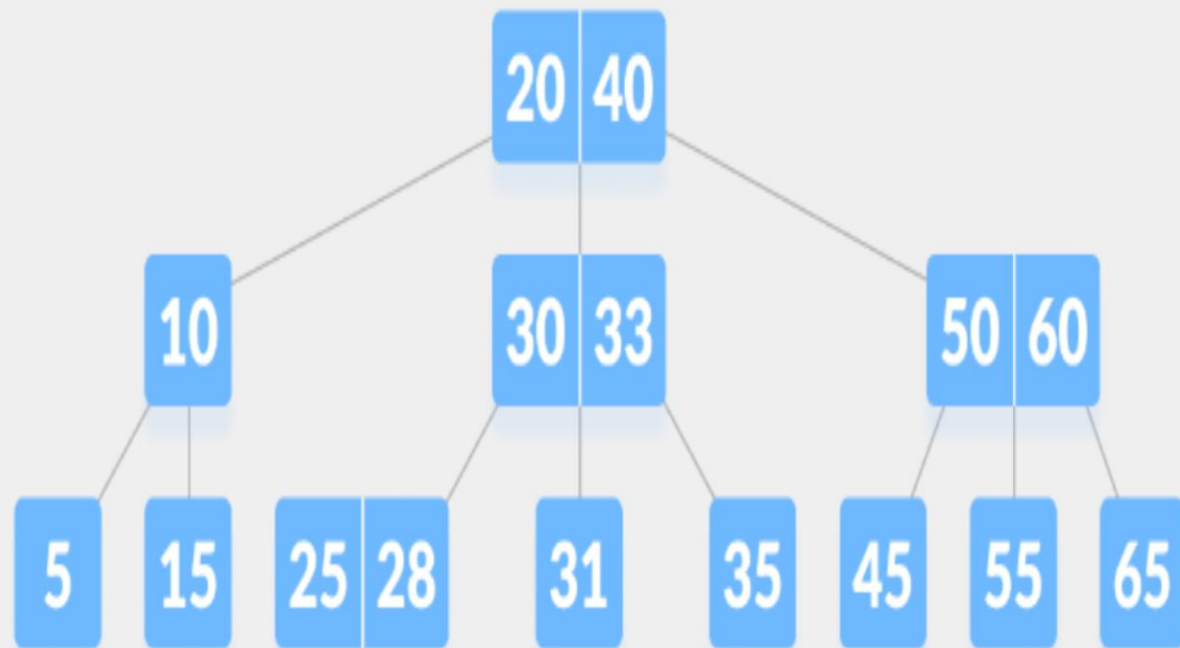
ต้นไม้แดง-ดำ (Red-Black)

- ต้นไม้แดง — ดำเป็นต้นไม้ที่ปรับสมดุลในตัวเองได้อีกชนิดหนึ่ง สร้างกฎขึ้นมาเพื่อกำหนดให้แต่ละโนดต้องมีสีเป็นแดงหรือดำอย่างใดอย่างหนึ่งตามกฎหมาย เมื่อมีการเพิ่มหรือลบโนดในโครงสร้าง จะทำการปรับสีของต้นไม้ย่อยต่างๆ ให้ได้ตามกฎหมาย ด้วยการหมุนต้นไม้ย่อยไปทางซ้ายและขวาเพื่อให้เกิดความสมดุล รายละเอียดจะนำเสนอในบทต่อไป แสดงตัวอย่างภาพของต้นไม้ประเภทนี้ดังนี้

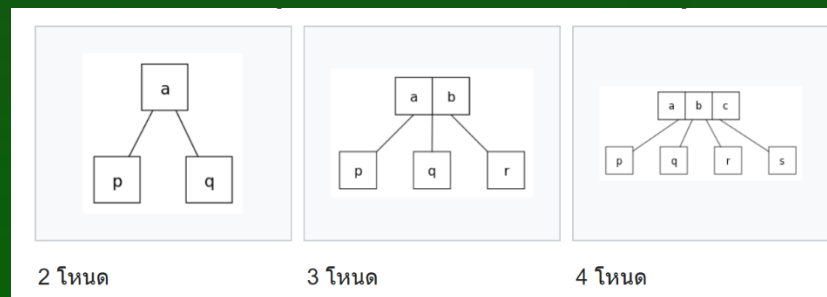
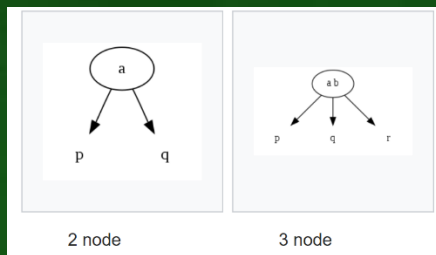


ต้นไม้บี (B-Tree)

- ต้นไม้บีเป็นต้นไม้ที่มีการปรับความสมดุลในตัวเองชนิดพิเศษ ซึ่งแต่ละโนดสามารถมีคีย์หรือค่าที่ระบุเป็นค่ารากได้มากกว่าหนึ่งค่า สามารถมีโนดลูกได้มากกว่าสองโนด ถือเป็นรูปแบบทั่วไปของต้นไม้ค้นหาแบบทวิภาค รู้จักกันในชื่อต้นไม้ **m-way** ที่มีความสมดุลสูง เกิดจากความต้องการเวลาเข้าถึงน้อย โดยทั่วไปนำไปประยุกต์ใช้ในสื่อบันทึกข้อมูล เช่น ฮาร์ดดิสก์ ที่ลดเวลาในการเข้าถึงข้อมูล เมื่อจำนวนชั้นของโครงสร้างต้นไม้ลดลง แสดงตัวอย่างภาพ ดังนี้

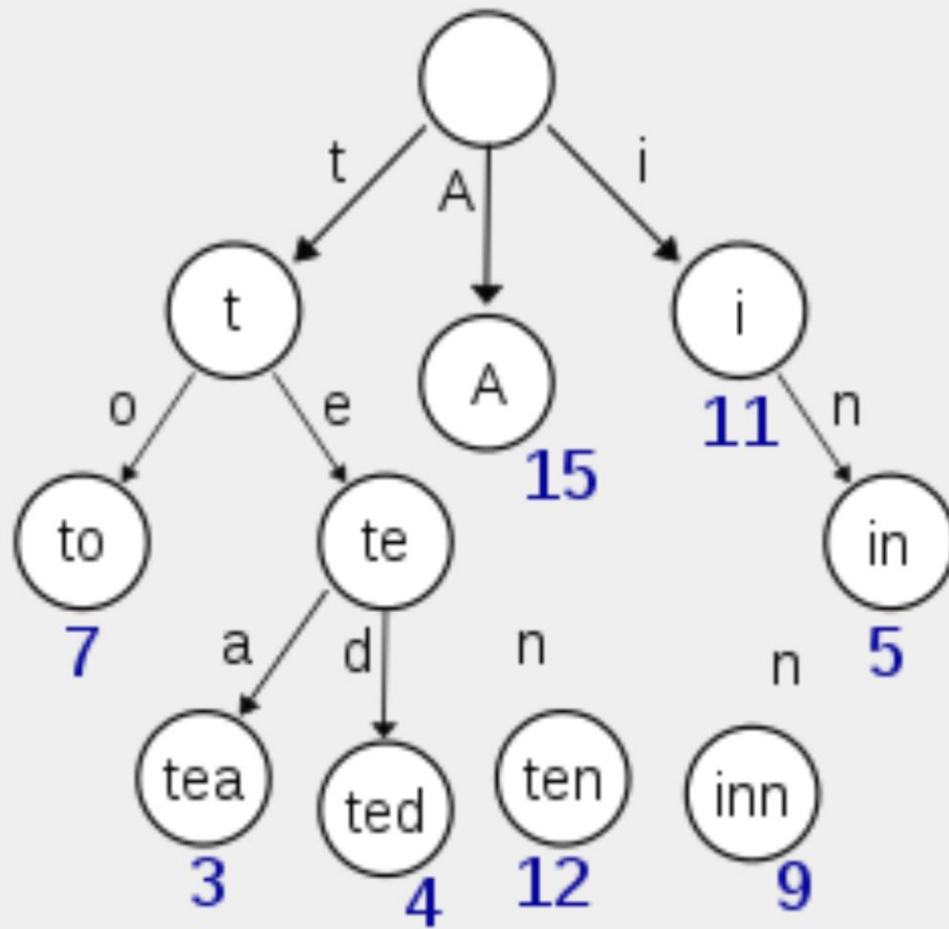


ต้นไม้ 2-3 และ 2-3-4 (2-3 Tree and 2-3-4 Tree)



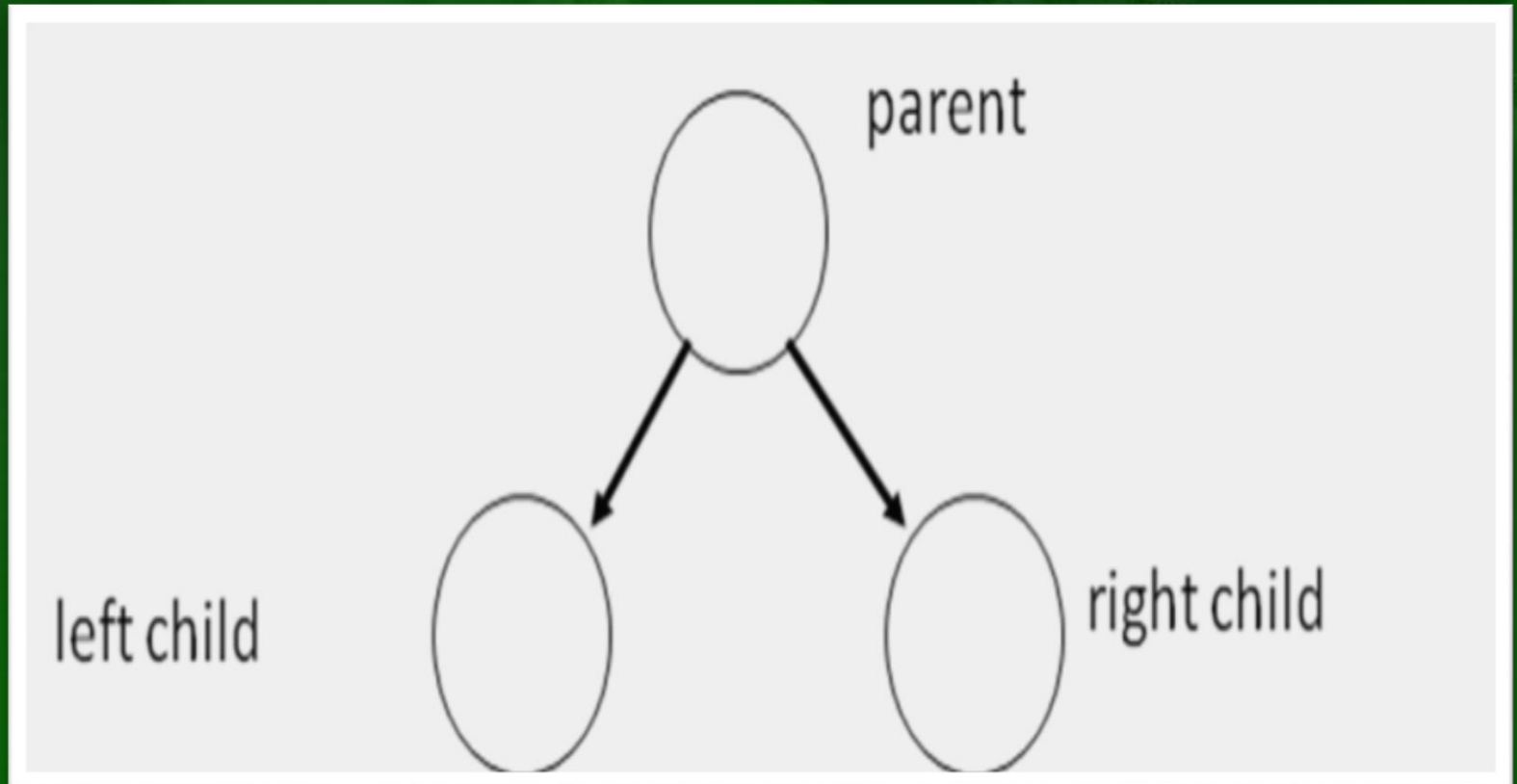
ทรี (Trie)

- ทรีเป็นโครงสร้างต้นไม้แบบหนึ่งที่แต่ละโนดและเส้นการเชื่อมใช้จัดเก็บตัวอักษร โดยทั่วไปจะเป็นสายอักขระ (**String**) เพื่อช่วยการจัดเก็บและค้นหาเปรียบคู่ในสายอักขระ ที่มากกว่าหนึ่งสายเป็นต้น ซึ่งในตำรานี้สำเสนอในบทที่ 13 โดยนำมาประยุกต์กับการค้นหาเปรียบคู่สายอักขระพหุแบบ (**Multiple String Matching Algorithm**) แสดงตัวอย่างดังภาพ



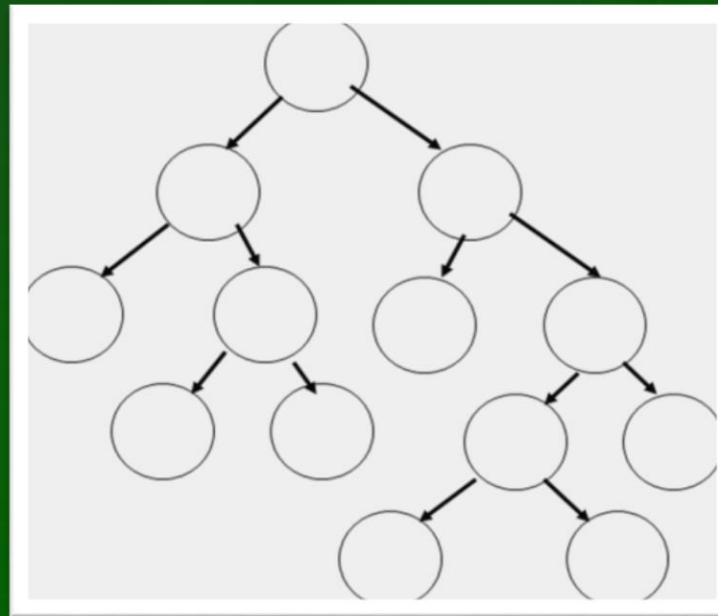
ต้นไม้ทวิภาค

- ต้นไม้ทวิภาค เป็นต้นไม้พื้นฐาน ที่ประกอบด้วยเซตของอิลิเมนต์เรียกว่าโนด สามารถเป็นได้ใน 2 ลักษณะคือ 1) ต้นไม้ว่าง (**Empty Tree**) คือต้นไม้ที่ไม่มีข้อมูล บางครั้งเรียกว่า **Null Tree** หรือ 2) ต้นไม้ที่ประกอบด้วย โนดราก (**Root**) และโนดที่เหลือของต้นไม้จะสร้างคู่ต้นไม้ย่อยซ้าย (**Left Sub-Tree**) และต้นไม้ย่อยขวา (**Right Sub-Tree**) อนุญาตให้แต่ละโนดมีโนดลูกได้ไม่เกิน 2 โนด
- จากคุณสมบัติดังกล่าว จึงทำการให้นิยามของต้นไม้เป็นลักษณะของเวียนเกิด (**Recursive**) เพราะหากพิจารณาต้นไม้ย่อยด้านซ้าย และต้นไม้ย่อยด้านขวา ก็สามารถระบุเป็นไปตามลักษณะของต้นไม้ทวิภาค เช่น ถ้าหากต้นไม้ **R** คือราก โดยมี **T1** คือ ต้นไม้ย่อยซ้าย **T2** คือ ต้นไม้ย่อยขวา ดังนี้
- **T1** จะประกอบด้วย **R_{T1}** และต้นไม้ย่อยซ้าย **T₁₋₁** และต้นไม้ย่อยขวา **T₂₋₂** โดยที่ต้นไม้ย่อยที่เกิดขึ้นสามารถมีต้นไม้ย่อยต่อเนื่องไปได้อีก
- **T2** จะประกอบด้วย **R_{T2}** และต้นไม้ย่อยซ้าย **T₂₋₁** และต้นไม้ย่อยขวา **T₂₋₂** โดยที่ต้นไม้ย่อยที่เกิดขึ้นสามารถมีต้นไม้ย่อยต่อเนื่องไปได้อีก
- หากพิจารณาลักษณะเช่นนี้จะเป็นการเวียนเกิดต่อเนื่องกันไป ต้นไม้ทวิภาค มีโนดลูกได้ไม่เกิน 2 ดังนั้น จะมีโนดแม่ (**Parent**) โนดลูกด้านซ้าย และโนดลูกด้านขวา (**Left and Right Child**)

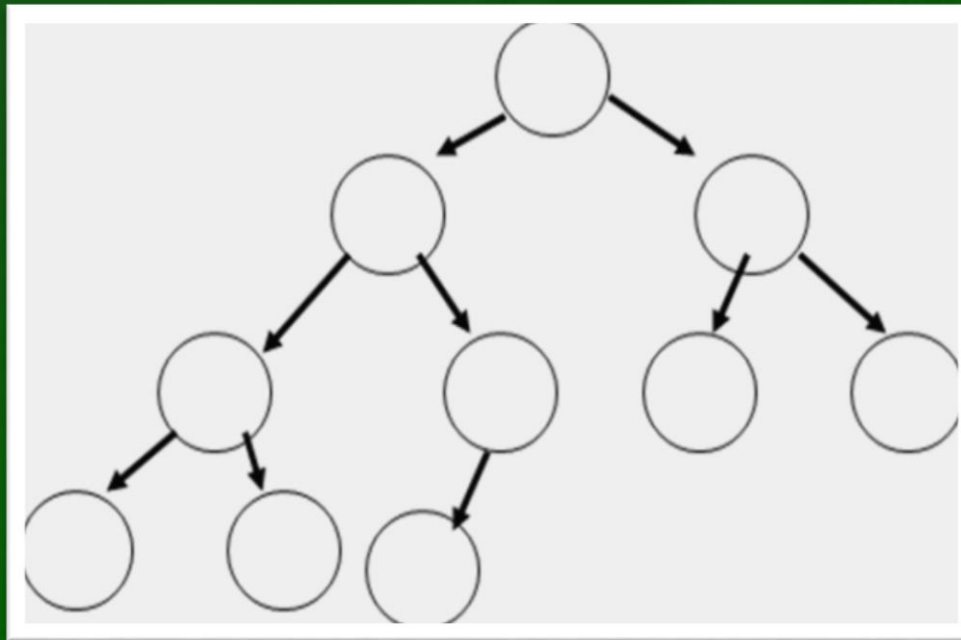


ประเภทของต้นไม้ที่วิภาค

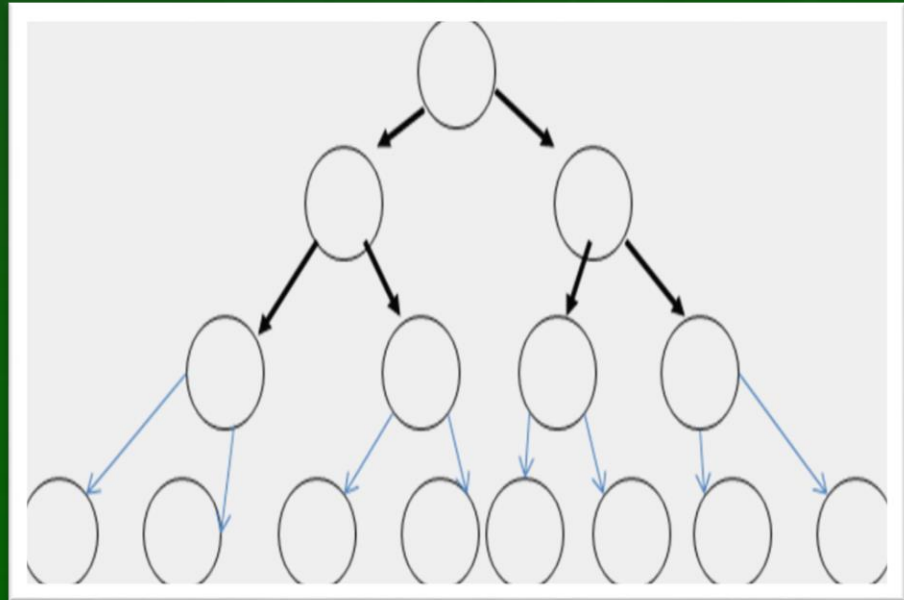
- ต้นไม้ทวิภาคแบบเต็ม (Full Binary Tree)
- คุณสมบัติ คือ ต้นไม้ทวิภาคที่มีโหนดลูก 2 หรือ 0 โหนด



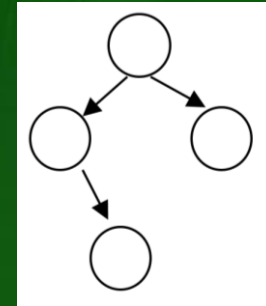
- ต้นไม้ทวิภาคแบบสมบูรณ์ (Complete Binary Tree)
- ต้นไม้ทวิภาคที่ทุกๆ ระดับจะมีจำนวนลูกครบ 2 โหนด ยกเว้นระดับลึกสุดในโนดสุดท้าย



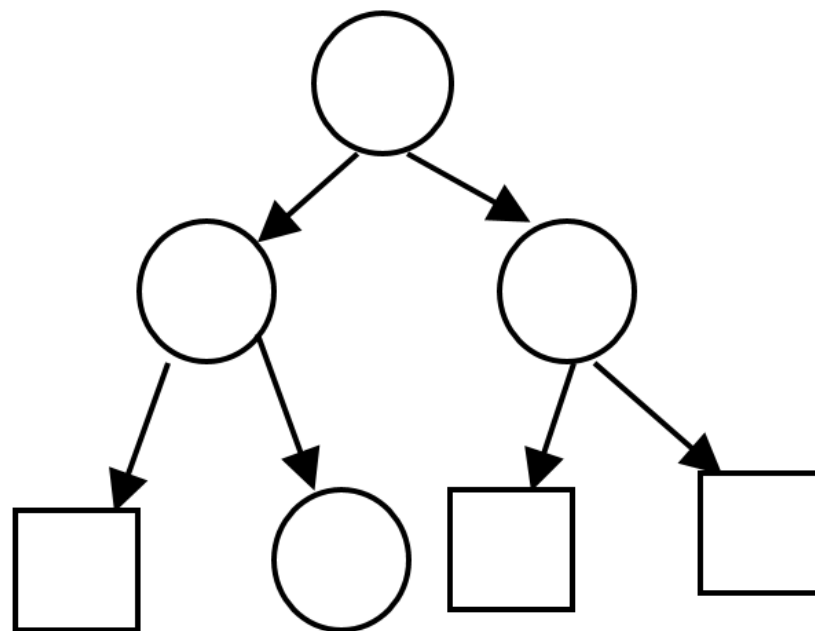
- ต้นไม้ทวิภาคแบบสมบูรณ์แบบ (Perfect Binary Tree)
- ต้นไม้ทวิภาคที่มีโนดลูกครบ 2 โนด ทุกระดับชั้น ซึ่งเป็นต้นไม้ที่ต้องการให้เป็นมากที่สุด แสดงดังภาพ



- ต้นไม้ทวิภาคแบบขยาย (**Extended Binary Tree**)
- เนื่องจากต้นไม้ทวิภาคมักจะไม่อยู่ในรูปแบบของต้นไม้สมบูรณ์แบบ ทำให้การเข้าถึงข้อมูลไม่มีประสิทธิภาพ ยกตัวอย่างต้นไม้ดังภาพ



- ดังนั้นการทำให้ต้นไม้เป็นแบบต้นไม้ทวิภาคสมบูรณ์แบบจึงทำให้เข้าถึงข้อมูลได้อย่างมีประสิทธิภาพ แต่เพียงโนดที่เพิ่มเข้าไปนั้น เป็นโนดจำลอง (**Pseudo Node**) เพื่อเกิดการท่องเข้าถึงได้ แต่ไม่ใช่ข้อมูลจริง ตัวอย่างตามแนวคิดนี้ ดังนี้



รูปที่ 7.17 ตัวอย่างต้นไม้ทวิภาคแบบขยาย

ความแตกต่างระหว่าง ต้นไม้ทั่วไป ต้นไม้ทวิภาค และต้นไม้ค้นหาทวิภาค

ความแตกต่างระหว่างต้นไม้ทั่วไปกับต้นไม้ทวิภาคดังนี้

- 1) ดีกรีในดของต้นไม้ทวิภาค มีได้โนดได้ไม่เกิน 2 แต่ต้นไม้ปกติสามารถมีดีกรี (เส้นเชื่อมออก) ได้ไม่จำกัด
- 2) ต้นไม้ทวิภาคอาจเป็นต้นไม้ว่างก็ได้ แต่ต้นไม้อื่นๆ ไม่อนุญาตให้เป็นต้นไม้ว่าง
- 3) ต้นไม้ทั่วไป ข้อมูลอาจไม่จัดเรียง ต้นไม้ทวิภาค ข้อมูลอาจไม่จำเป็นต้องจัดเรียง แต่ต้นไม้ค้นหาทวิภาค จะมีการจัดเรียงเรียบร้อยแล้ว โดยใช้โนดรากเป็นตัวแบ่งข้อมูลที่อยู่ฝั่งซ้ายของรากจะต้องน้อยกว่าหรือเท่ากับราก ฝั่งขวาต้องมากกว่าโนดราก
- ดังนั้น ในบทนี้หากกล่าวถึงอัลกอริทึมในการจัดการโครงสร้างต้นไม้ทวิภาค จะหมายถึง ต้นไม้ค้นหาทวิภาค โดยการจัดเรียงข้อมูลแล้วเท่านั้น

การเขียนรหัสโปรแกรมของโครงสร้างต้นไม้ด้วยภาษาอาวา

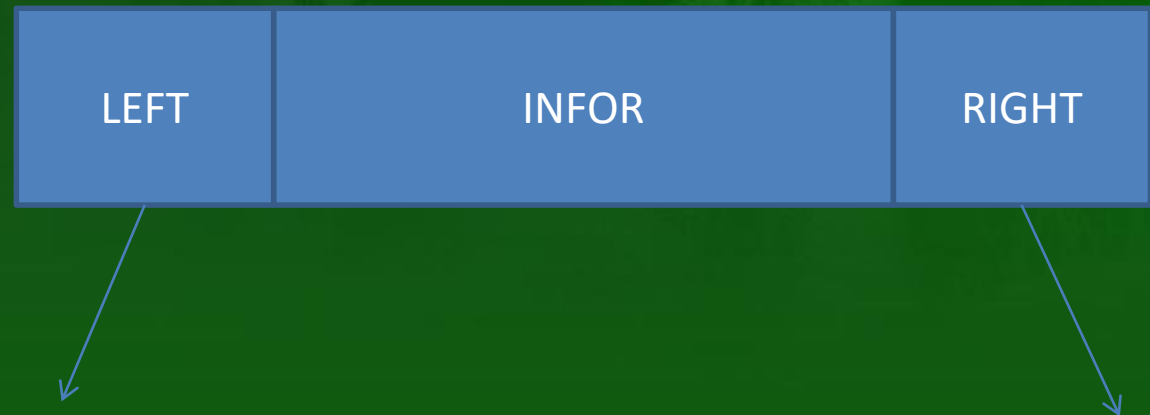
- นิยามที่ **7.2** กำหนดให้
- **INFO** คือ ข้อมูลใดๆ ที่บรรจุอยู่ในโนดของต้นไม้ค้นหาทวิภาค
- **LEFT** คือ ตัวชี้ (Pointer) ที่ชี้ไปยังโนดทางซ้าย
- **RIGHT** คือ ตัวชี้ (Pointer) ที่ชี้ไปยังโนดทางขวา
- **ROOT** คือ ตัวชี้ (Pointer) ที่ชี้ไปยังโนดรากของต้นไม้ค้นหาทวิภาค

ไบนารีทรี (Binary Tree)

- ไบนารีทรี **T** เป็นทรีที่ประกอบด้วยเซตของอิลิเมนต์ที่เรียกว่า โหนด มีอยู่ 2 ลักษณะ คือ
 - ทรีว่าง ไม่มีข้อมูล เรียกว่า **null tree, empty tree**
 - หรือทรี ประกอบด้วย โหนดราก (**Root**) และโหนดที่เหลือของทรี จะสร้างคู่ทรีย่อยออกไปทางซ้ายและขวา (**T1** และ **T2**) โดยกำหนดให้มีโหนดลูกได้ ไม่เกิน 2 โหนด

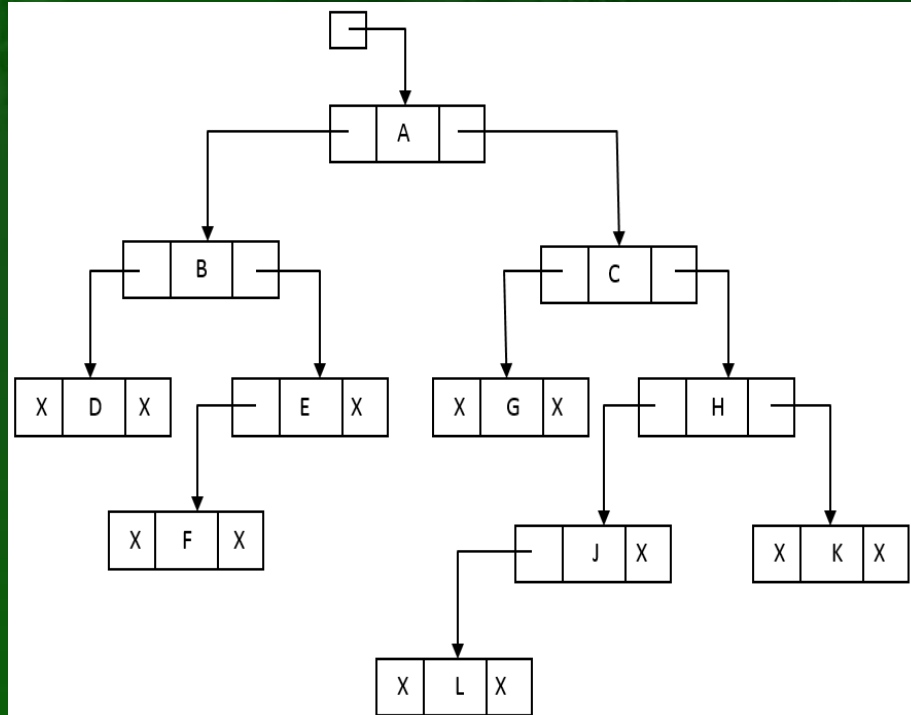
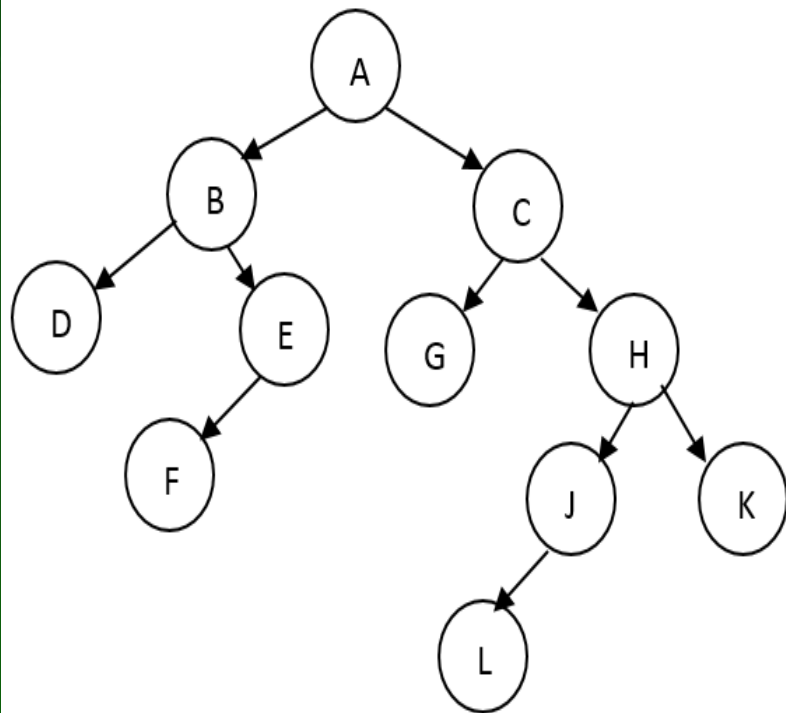
แนวคิดการสร้างโนดในหน่วยความจำ

- ROOT
- INFOR
- LEFT
- RIGHT



ตัวอย่างการอิมพลีเมนต์โนด

- public class Node {
- Object INFOR;
- Node LEFT=null;
- Node RIGHT=null;
- }



การทดลอง

- Node A = new Node(); A.INFOR = "A";
- Node B = new Node(); B.INFOR = "B";
- Node C = new Node(); C.INFOR = "C";
- Node D = new Node(); D.INFOR = "D";
- Node E = new Node(); E.INFOR = "E";
- Node F = new Node(); F.INFOR = "F";
- Node G = new Node(); G.INFOR = "G";
- Node H = new Node(); H.INFOR = "H";
- Node J = new Node(); J.INFOR = "J";
- Node K = new Node(); K.INFOR = "K";
- Node L = new Node(); L.INFOR = "L";

•

- Node ROOT; //กำหนดตัวแปร ROOT

- //เชื่อมโยงการใช้งานตามรูปภาพที่ 7-6

- A.LEFT = B; A.RIGHT = C;

- B.LEFT = D; B.RIGHT = E;

- E.LEFT = F;

- C.LEFT = G; C.RIGHT = H;

- H.LEFT = J; H.RIGHT = K;

- J.LEFT = L;

•

- ROOT=A; // กำหนดค่า Root ชี้ไปยังโนด A

การดำเนินการกับไบนาริทีรี

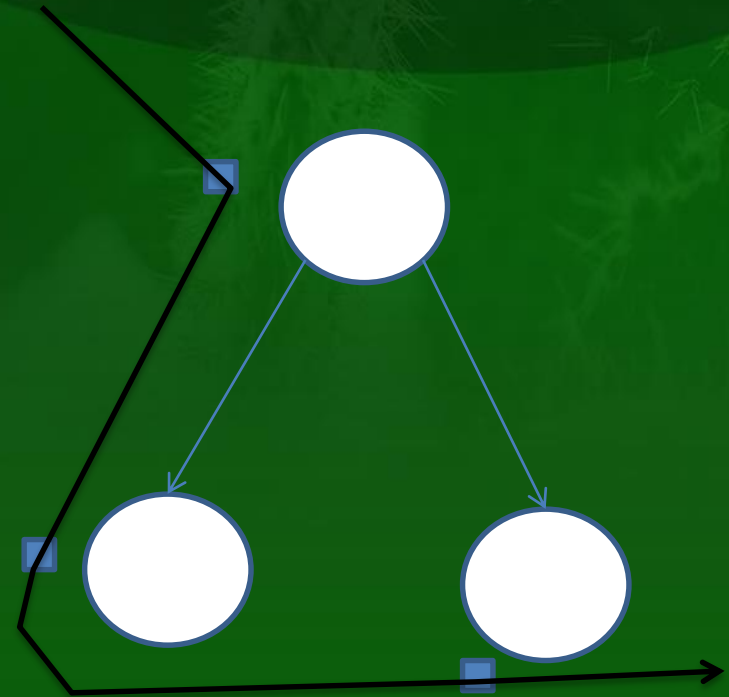
- การท่อนทีรี
- การค้นหาข้อมูล
- การแทรกข้อมูล
- การลบข้อมูล

การท่องทรี (Traversing Binary Tree)

- พรีออร์เดอร์ (Pre-Order)
- อินออร์เดอร์ (In-Order)
- पोสออร์เดอร์ (Post-Order)

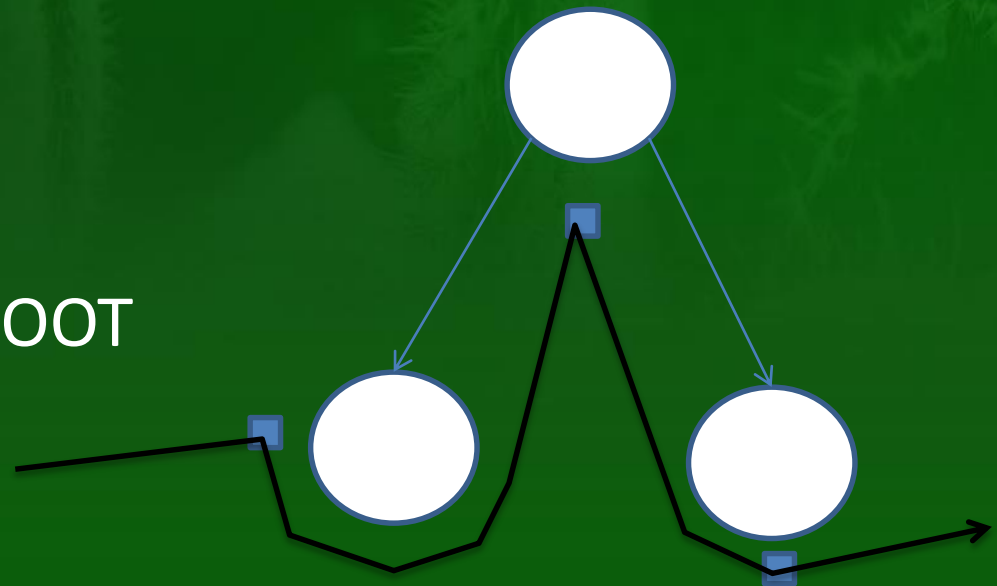
Pre-Order

- ลำดับการเข้าถึง
- เข้าที่ ROOT
- ท่องไปด้านซ้าย LEFT ของ ROOT
- ท่องไปด้านขวา RIGHT ของ ROOT
- ROOT-LEFT-RIGHT



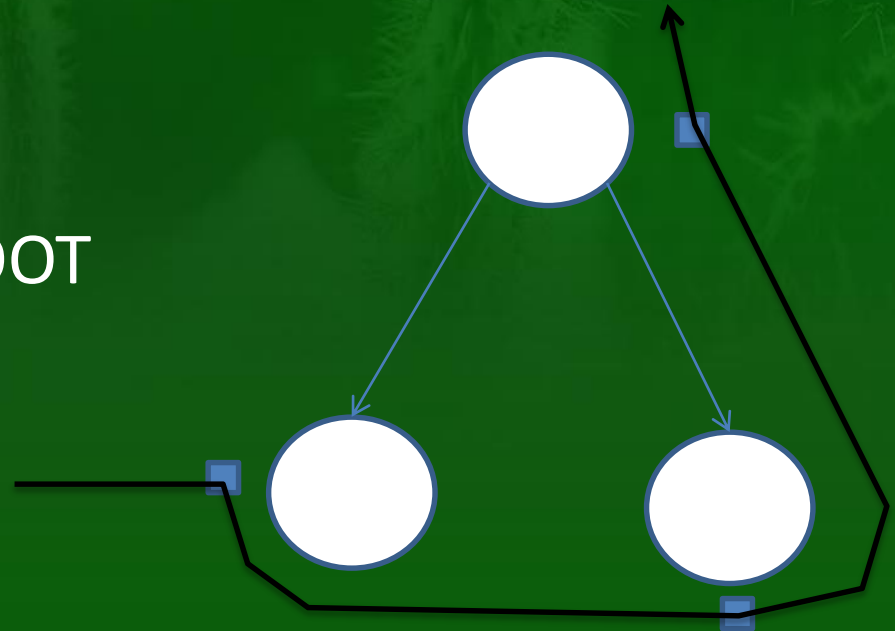
In-Order

- ลำดับการเข้าถึง
- เข้าที่ ซ้าย LEFT ของ ROOT
- ท่องไป ROOT
- ท่องไปด้านขวา RIGHT ของ ROOT
- LEFT-ROOT-RIGHT



Post-Order

- ลำดับการเข้าถึง
- เข้าที่ ซ้าย **LEFT** ของ **ROOT**
- ท่องไปด้านขวา **RIGHT** ของ **ROOT**
- ท่องไป **ROOT**
- **LEFT-RIGHT-ROOT**



ตัวอย่างการทอแบบฟรีออร์เคอร์

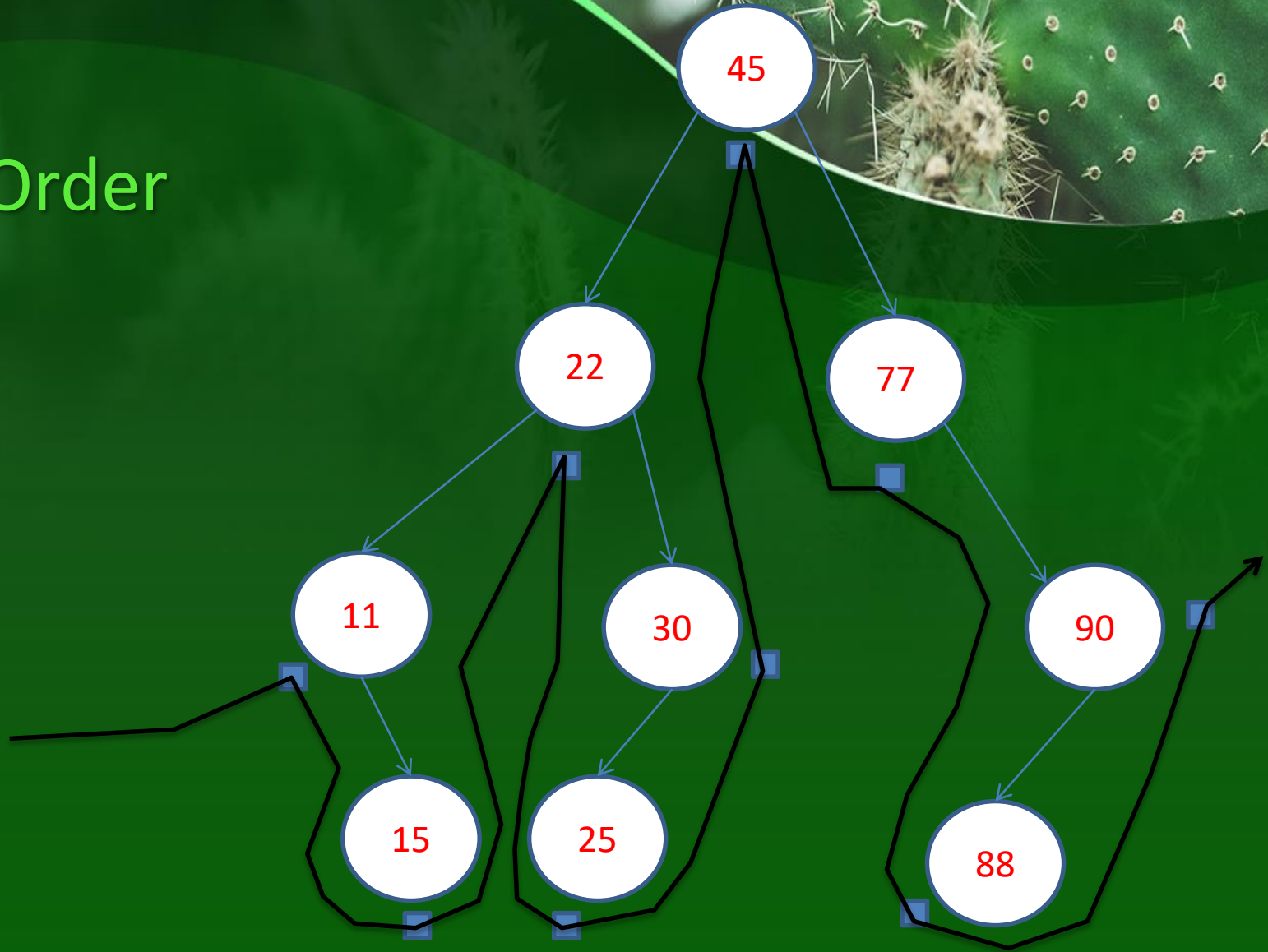
- ROOT-LEFT-RIGHT

Pre-Order



- 45, 22, 11, 15, 30, 25, 77, 90, 88

In-Order

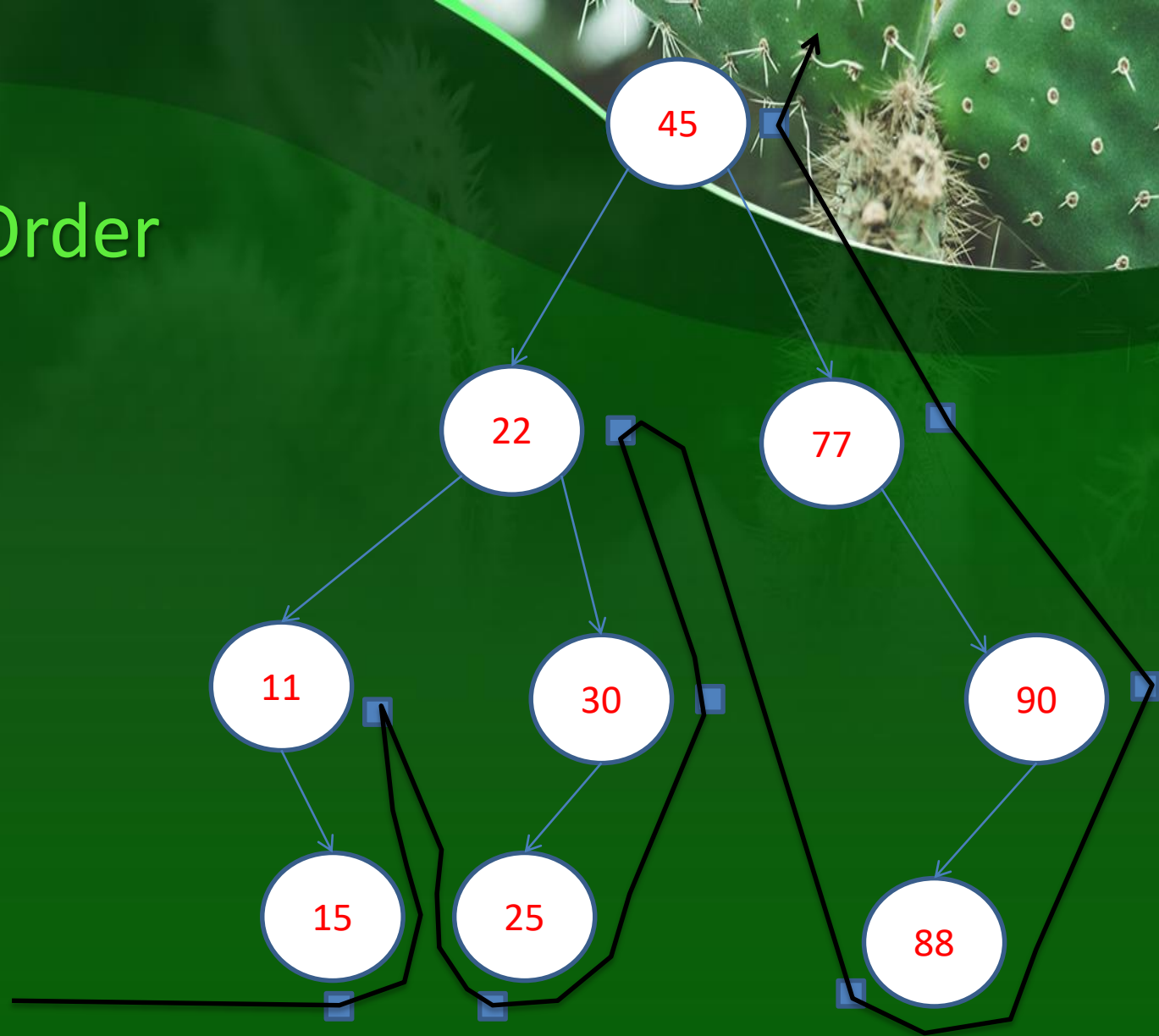


- 11,15,22,30,25,45,77,88,90

ตัวอย่างการทอแบบโพสอ์เคอร์

- LEFT-RIGHT-ROOT

Post-Order



- 15,11,23,30,32,88,90,77,45

ตัวอย่างการอิมพลีเมนต์

- อิมพลีเมนต์โดยใช้สแต็กเป็นตัวดำเนินการ

Pre-Order Algorithm

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. เริ่มจากเก็บค่า
 - 1.1 NULL ลงใน STACK
 - 1.2 กำหนด PTR = ROOT
2. ทำซ้ำขั้นตอน 3 และ 4 จนกระทั่ง PTR = NULL ซึ่งเท่ากับ while PTR $\neq$ NULL
3. กระทำจาก PTR ลงไปทางซ้ายสุด โดยประมวลผลในแต่ละโนดที่เข้าถึง พร้อมกับเก็บโนดลูกด้านขวาลงสแตก STACK ดำเนินการเช่นนี้ต่อไปนี้ จนกระทั่งไม่มีโนดลูกด้านซ้าย (ค่า PTR จะเปลี่ยนไปจากการกำหนด PTR = LEFT (PTR)) และการเข้าถึงข้อมูลสิ้นสุดเมื่อ LEFT [PTR] = NULL
4. ดึงค่าจากสแตก STACK มา กำหนดให้กับ PTR หาก PTR $\neq$ NULL แล้วกลับไปทำในขั้นตอน 3 หากเป็น อย่างอื่นให้จบงาน

Algorithm 7.1: PREORDER

Input : INFO, LEFT, RIGHT, ROOT, STACK, TOP

Output : All nodes in BST are traversed

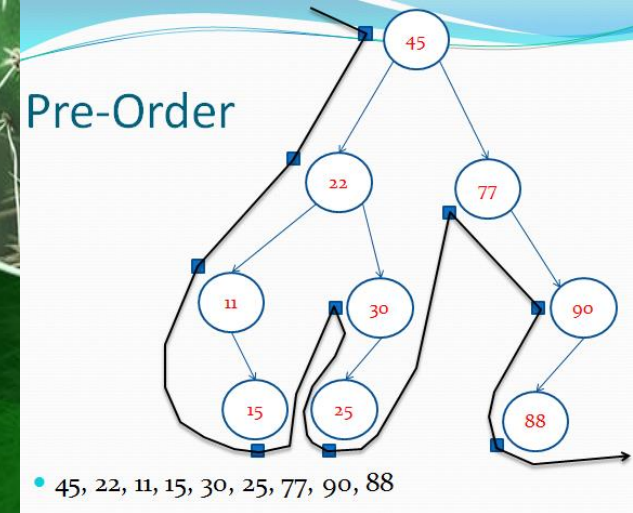
1. TOP := 1
2. STACK[1] := NULL
3. PTR := ROOT
4. While PTR $\neq$ NULL Do
5. Process INFO[PTR] //ท่องโนด
6. If RIGHT[PTR] $\neq$ NULL Then
 //นำโนดเก็บลง Push Stack
7. TOP := TOP + 1 and STACK[TOP] :=
 RIGHT[PTR]
8. End of If
9. If LEFT [PTR] $\neq$ NULL Then
 //ตรวจสอบว่ามีโนดซ้ายหรือไม่
10. PTR := LEFT [PTR]. //ถ้ามีโนด
11. Else
 //กรณีไม่มีโนดซ้าย Pop โหนดจาก STACK
12. PTR := STACK[TOP] and TOP := TOP - 1.
13. End of If
14. End of While
15. Return

Pre-Order



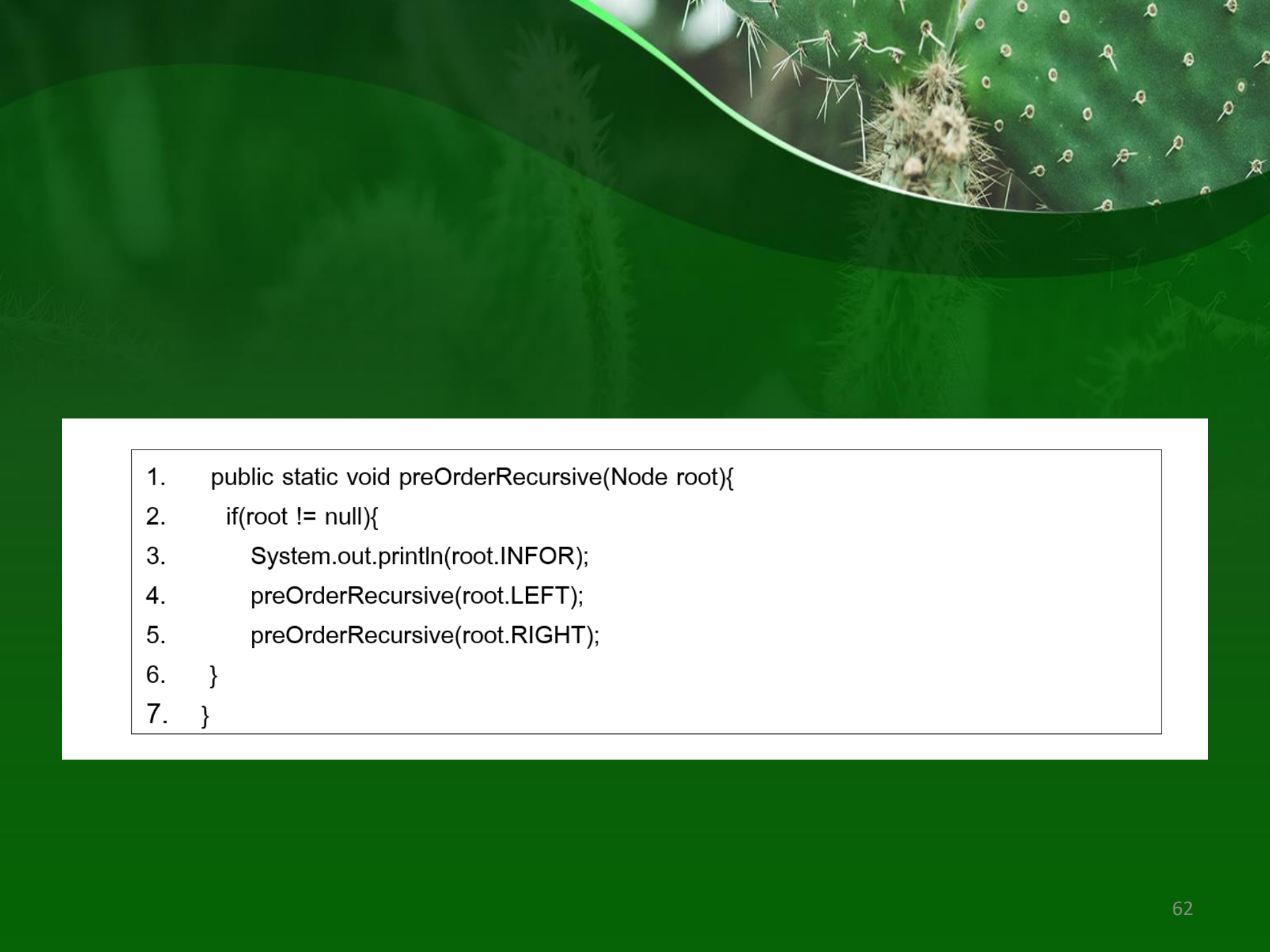
- 45, 22, 11, 15, 30, 25, 77, 90, 88

- Push NULL, Stack: NULL
- Process 45, push 77 Stack: NULL, 77
- Process 22 push 30 Stack: NULL, 77, 30
- Process 11 push 15 Stack: NULL, 77, 30, 15
- Pop 15 Process 15 Stack: NULL, 77, 30
- Pop 30, Process 30 Stack: NULL, 77
- Process 25 Stack: NULL, 77
- Pop 77 Process 77, push 90 Stack: NULL, 90
- Pop 90 Process 90 Stack: NULL
- Process 88 Stack: NULL
- Pop NULL Exit
- Traversing Lists: 45, 22, 11, 15, 30, 25, 77, 90, 88



ตัวอย่าง 7.7 แสดงการเขียนโปรแกรมประกอบอัลกอริทึมการท่องแบบพรีออร์เดอร์ดังนี้

```
1. public static void PREORDER(Node root){
2.     int TOP=1;
3.     STACT[1]=null;
4.     Node PTR=root;
5.     while(PTR!=null){
6.         System.out.println(PTR.INFOR); //Apply PROCESS to INFOR[PTR]
7.         if(PTR.RIGHT != null){
8.             TOP++;
9.             STACT[TOP] = PTR.RIGHT;
10.        }
11.        if(PTR.LEFT != null){
12.            PTR = PTR.LEFT;
13.        }else
14.        {
15.            PTR = STACT[TOP];
16.            TOP--;
17.        }
18.    } //end of while loop
19. }
20. }
```



```
1.  public static void preOrderRecursive(Node root){
2.      if(root != null){
3.          System.out.println(root.INFOR);
4.          preOrderRecursive(root.LEFT);
5.          preOrderRecursive(root.RIGHT);
6.      }
7.  }
```

การทดลอง

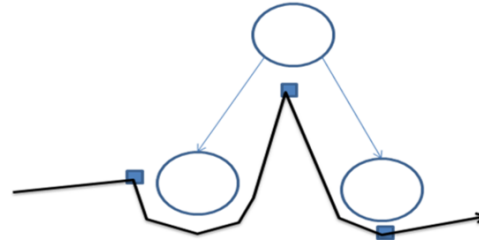
ตัวอย่างการท่องอินอ์เคอร์

- LEFT-ROOT-RIGHT

7.4.2 การท่องต้นไม้ค้นหาทวิภาคแบบอินเนอร์เดอร์

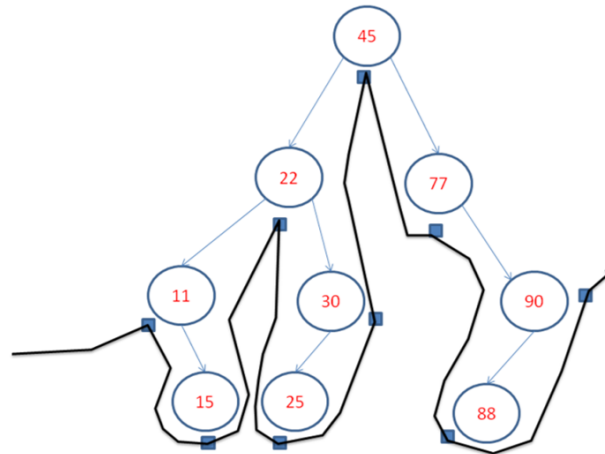
แนวคิดลำดับการท่อง คือ เริ่มจากท่องที่โนดซ้ายไปหารากและไปโนดขวาลำดับการเข้าถึง

1. เข้าที่ ซ้าย LEFT ของ ROOT
2. ท่องไป ROOT
3. ท่องไปด้านขวา RIGHT ของ ROOT



รูปที่ 7.24 แสดงแนวคิดการท่องต้นไม้ค้นหาทวิภาคแบบอินเนอร์เดอร์

ตัวอย่างการท่องแบบโพสออร์เดอร์ จากรูปที่ 7.25 ดังนี้



11, 15, 22, 25, 30, 45, 77, 88, 90

รูปที่ 7.25 แสดงตัวอย่างท่องต้นไม้ค้นหาทวิภาคแบบอินเนอร์เดอร์

กำหนดตัวแปร INFO , LEFT , RIGHT , ROOT , TOP , STACK เพื่อท่องทุกโนดในต้นไม้ ขั้นตอนดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. เริ่มต้นกำหนด NULL ลงใน STACK และกำหนด PTR = ROOT
2. ทำตามขั้นตอน 3 และ 4 จนกว่า NULL จะถูกดึงออกมาจาก STACK
3. กระทำลงไปทางซ้ายจากราก PTR เก็บแต่ละโนด N ลงใน STACK และหยุดเมื่อโนด N ไม่มีโนดลูกด้านซ้าย
4. [ทำย้อนกลับ] ดึงค่าจาก STACK และประมวลผล ถ้าพบว่าเป็น NULL ก็จบการทำงาน ถ้าโนด N มีโนดลูกด้านขวาที่ถูกประมวลผล กำหนด PTR = R (N) (โดยใช้การกำหนด) PTR := RIGHT [PTR] และกลับไปทำในขั้นตอน 3

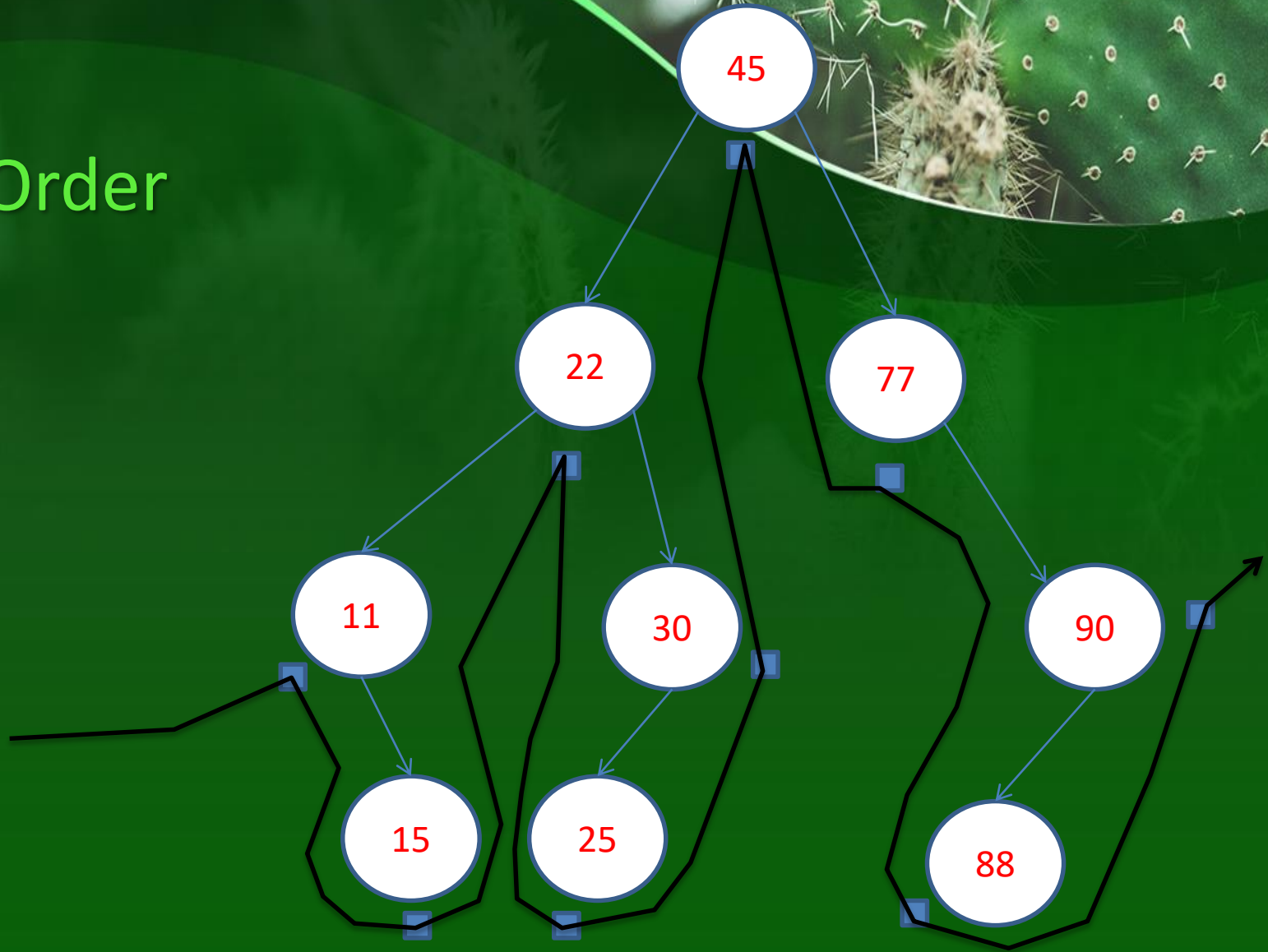
Algorithm 7.2: INORDER

Input : INFO , LEFT , RIGHT , ROOT

Output : All nodes in BST are traversed

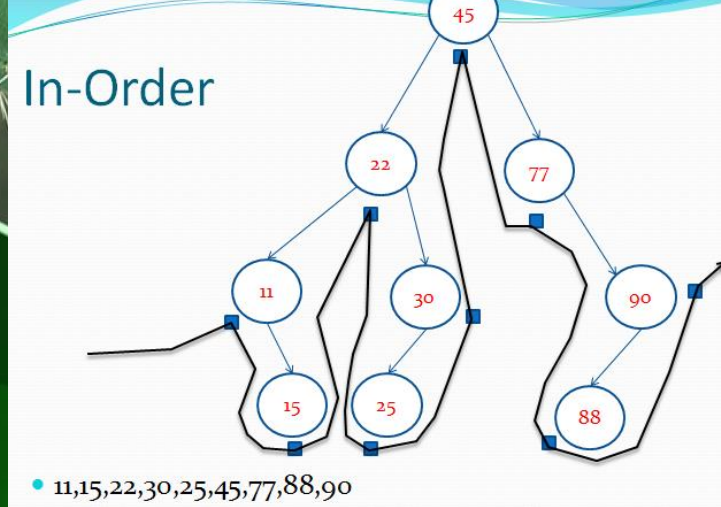
1. TOP := 1, STACK[1] := NULL and PTR := ROOT.
2. While PTR $\neq$ NULL Do
 - 2.1 TOP := TOP+1 and STACK[TOP] := PTR. // เก็บโนด
 - 2.2 PTR := LEFT[PTR].
//อัปเดต PTR
3. End of While
4. PTR := STACK[TOP] and TOP := TOP - 1
// Pops โหนดจาก STACK
5. While Step 6 to 8 while PTR $\neq$ NULL Do // ทำการย้อนกลับ
6. Process INFO[PTR] //ท่องโนด
7. If RIGHT[PTR] $\neq$ NULL Then
// มีโนดขวาหรือไม่
 - 7.1 PTR := RIGHT [PTR].
 - 7.2 Go to Step 2. //เก็บโนด
8. End of If
9. PTR=STACK[TOP] and TOP := TOP-1
//Pop โหนด
- End of While
10. Return

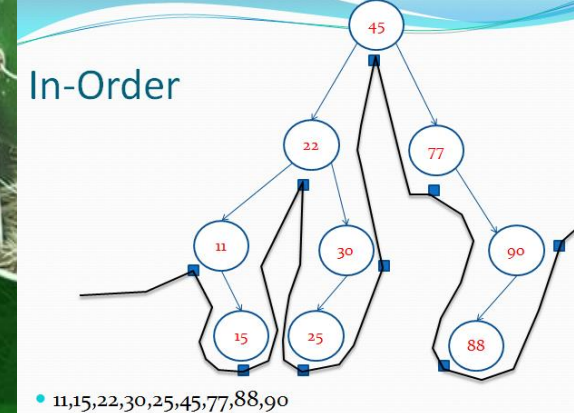
In-Order



- 11,15,22,25. 30,45,77,88,90

- Push NULL, Stack: NULL
- Push 45 Stack: NULL, 45 มีโนดซ้าย ต้อง Push ต่อ
- Push 22 Stack: NULL , 45, 22 มีโนดซ้าย ต้อง Push ต่อ
- Push 11 Stack: NULL , 45, 22, 11 ไม่มีโนดซ้าย หยุด Push
- Pop 11 Process 11 Stack: NULL , 45, 22 มีโนดขวา หยุด Pop ดำเนินการ Push ต่อ
- Push 15 Stack: NULL , 45, 22, 15 ไม่มีโนดซ้าย หยุด Push
- Pop 15 Process 15, Stack: NULL , 45, 22 ไม่มีโนดขวา
- Pop 22 Process 22 Stack: NULL , 45 มีโนดขวา หยุด Pop
- Push 30 Stack: NULL , 45,30 มีโนดซ้าย ต้อง Push ต่อ





- Push 25 Stack: NULL , 45,30, 25 ไม่มีโนดซ้าย หยุด Push
- Pop 25 Process 25 Stack: NULL , 45,30 ไม่มีโนดขวา
- Pop 30 Process 30 Stack: NULL , 45 ไม่มีโนดขวา
- Pop 45 Process 45 Stack: NULL มีโนดขวา หยุด Pop
- Push 77 Stack: NULL, 77 ไม่มีโนดซ้าย หยุด Push
- Pop 77 Process 77 Stack: NULL มีโนดขวา หยุด Pop
- Push 90 Stack: NULL,90 มีโนดซ้าย Push ต่อ
- Push 88 Stack: NULL,90,88 ไม่มีโนดซ้าย หยุด Push
- Pop 88 Process 88 Stack: NULL,90 ไม่มีโนดขวา Pop ต่อ
- Pop 90 Process 90 Stack: NULL ไม่มีโนดขวา Pop ต่อ
- Pop NULL จบงาน

การทดลอง

ตัวอย่าง 7.9 แสดงการเขียนโปรแกรมตามแนวคิดการท่องอินอเดอร์เป็นดังนี้

```
1. public static void INORDER(Node root){
2.     PTR=root;
3.     Step2();
4.     PTR = STACK[TOP]; TOP--;
5.     while(PTR != null){           //step 4
6.         System.out.println(PTR.INFOR); //step5
7.         if(PTR.RIGHT != null){ //step 6
8.             PTR = PTR.RIGHT;
9.             Step2();
10.        }
11.    PTR = STACK[TOP]; TOP--;
12.    }
13. }
14. public static void Step2(){
```

```
15. while(PTR != null){
16.     TOP++; STACK[TOP] = PTR;
17.     PTR = PTR.LEFT;
18. } //end of while loop
19. }
```

นอกจากนี้ยังสามารถเขียนโปรแกรมให้เป็นแบบเวียนเกิด แสดงรหัสโปรแกรมการท่องต้นไม้ค้นหาทวิภาคแบบอินออร์เดอร์ได้ดังนี้

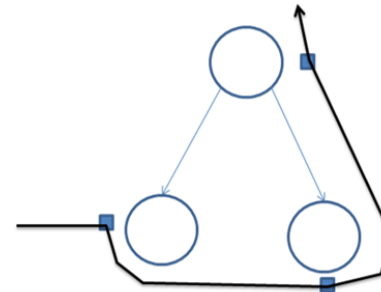
```
1. public static void inOrderRecursive(Node root){
2.     if(root != null){
3.         inOrderRecursive(root.LEFT);
4.         System.out.println(root.INFOR);
5.         inOrderRecursive(root.RIGHT);
6.     }
7. }
```

Post Order Algorithm

7.4.3 การท่องต้นไม้ค้นหาวิภาคแบบโพสตร์เดอร์

แนวคิดการท่องแบบอินอเดอร์มีลำดับการเข้าถึงดังนี้

1. เข้าที่ ซ้าย LEFT ของ ROOT
2. ท่องไปด้านขวา RIGHT ของ ROOT
3. ท่องไป ROOT



รูปที่ 7.27 แสดงแนวคิดแบบโพสตร์เดอร์

แสดงตัวอย่างการท่องตามแนวคิดโพสตร์เดอร์

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. เริ่มต้นเก็บ NULL ลงใน STACK และกำหนด $PTR := ROOT$
2. ทำขั้นตอนต่อไปนี้อย่างซ้ำ 3 และ 4 จนกระทั่งค่า NULL ถูกดึงออกมาจาก STACK
3. ทำลงไปทางซ้ายจากราก PTR ที่โหนด N ของทางผ่านเก็บ N ลงใน STACK และถ้า N มีโหนดลูกด้านขวา $R(N)$ เก็บ $-R(N)$ ลงใน STACK
4. [ทำย้อนกลับ] ดึงค่าในโหนดที่เป็นบวกออกจาก STACK ถ้าพบ NULL จบการทำงาน ถ้าค่าที่ดึงออกจาก STACK เป็นโหนดลบ นั่นคือ $PTR := -N$ กำหนด $PTR := N$ (โดยการกำหนด $PTR := PTR$) แล้วกลับไปทำในขั้นตอน 3 การประมวลผลจะทำกับค่าโหนดที่ดึงออกจาก STACK และเป็นบวกเท่านั้น

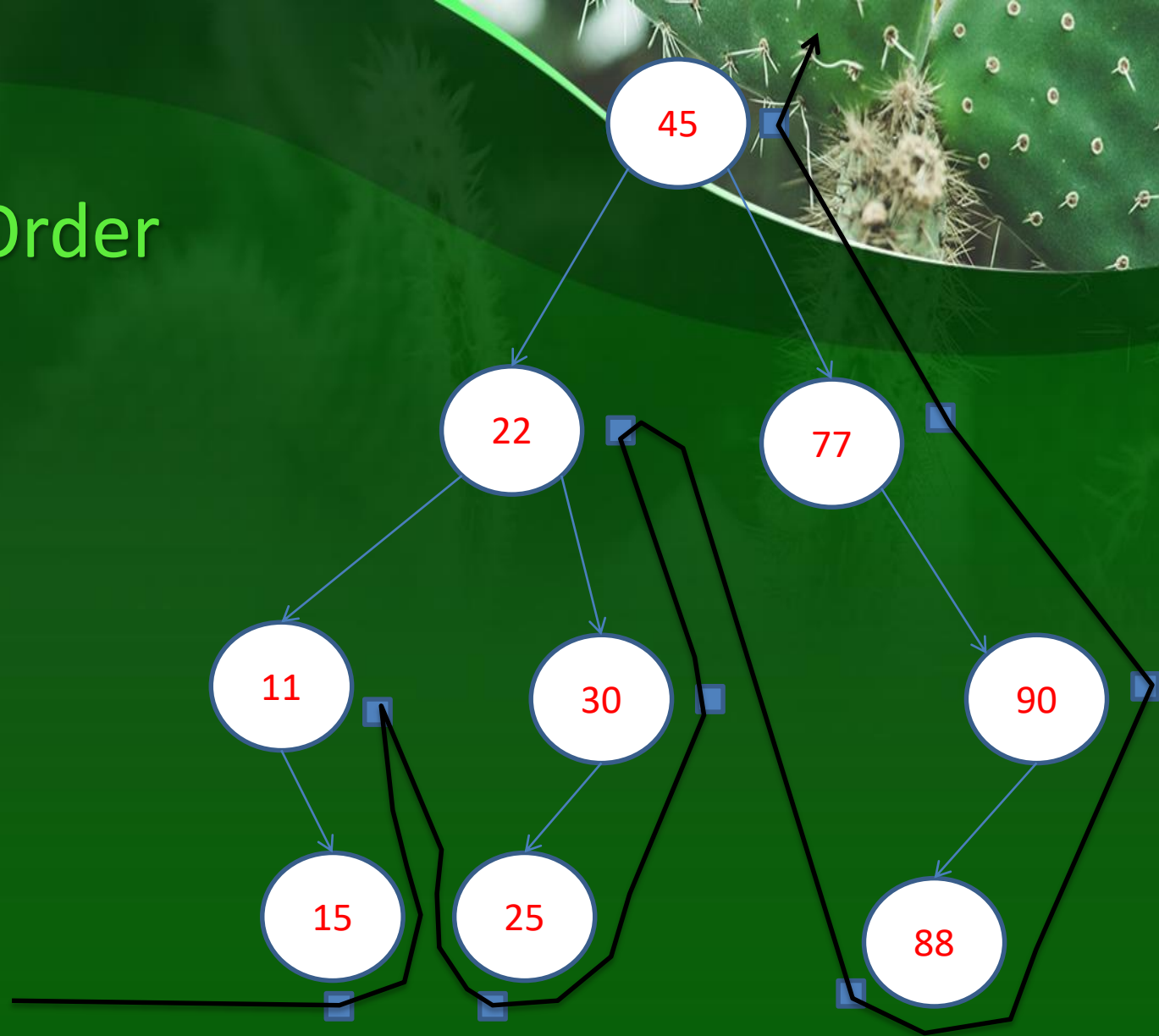
Algorithm 7.3 : POSTORDER

Input : INFO , LEFT , RIGHT , ROOT

Output : All nodes in BST are traversed

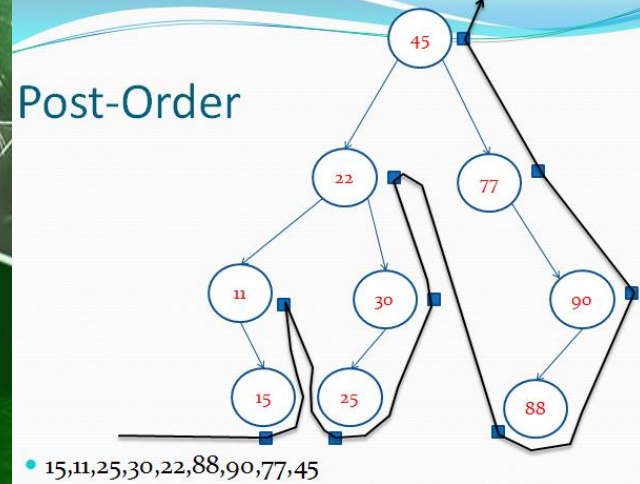
1. Set TOP := 1, STACK[1] := NULL and PTR := ROOT
2. While PTR $\neq$ NULL Do
3. TOP := TOP + 1 and STACK [TOP] := PTR
 // เก็บโนด PTR ลง STACK
4. If RIGHT [PTR] $\neq$ NULL Then //เก็บลง STACK
5. TOP := TOP + 1 and STACK[TOP] := -RIGHT[PTR]
 //เก็บโนดที่ให้ค่ากลับ
6. End of If
7. PTR := LEFT[PTR] //เลื่อนตัวชี้ไปยังซ้าย PTR
8. End of While // จบบรรทัดที่ 2
9. PTR := STACK[TOP] and TOP := TOP - 1
 // Pop โนดจาก STACK
10. While PTR > 0 Do //ตรวจสอบว่าโนดเป็นค่าปกติหรือไม่
- 10.1 Process INFO[PTR]. //ท่องโนด
- 10.2 PTR := STACK[TOP] and TOP := TOP -1.
 //Pop โนดจาก STACK
11. End of While
12. If PTR < 0 Then //ถ้าเป็นค่ากลับ ให้เปลี่ยนค่าและเก็บโนด
- 12.1 PTR := -PTR.
- 12.2 Go to Setp 2.
13. End of If
14. Return

Post-Order



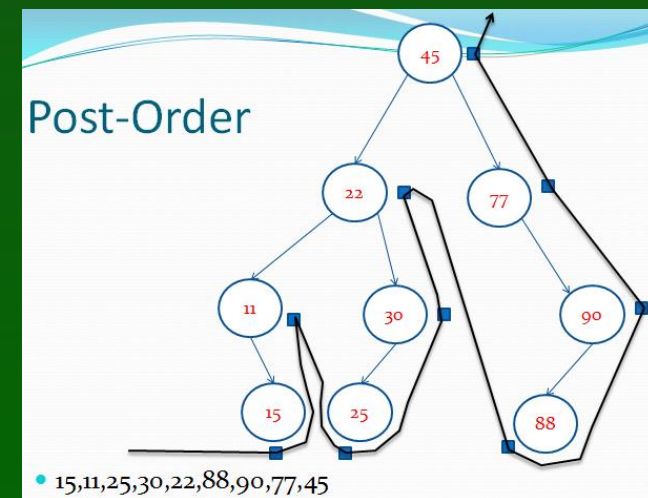
- 15,11,25,30,22,88,90,77,45

Post-Order



- Push NULL Stack:NULL
- Push 45, -77, 22 Stack:NULL, 45, -77, 22 มีโนดซ้าย ของ 22 Push ต่อ
- Push -30 , Push 11 Stack:NULL, 45, -77, 22, -30, 11 โนด 11 มีโนดขวา
- Push -15 Stack:NULL, 45, -77, 22, -30, 11, -15 ไม่มีโนดซ้าย หยุด Push
- Pop -15 กลับ เครื่องหมายเป็น 15 ดำเนินการ Push ซ้ำ
- Push 15 Stack:NULL, 45, -77, 22, -30, 11, 15 ไม่มีโนดซ้าย หยุด Push
- Pop 15 เครื่องหมาย เป็นลบ **Process 15** Stack:NULL, 45, -77, 22, -30, 11
- Pop 11 เครื่องหมาย เป็นลบ **Process 11** Stack:NULL, 45, -77, 22, -30
- Pop -30 เครื่องหมายเป็นลบ กลับเครื่องหมายเป็นบวก 30 ดำเนินการ Push ซ้ำ เริ่ม Push 30
- Push 35 Stack:NULL, 45, -77, 22, 30, 35 ไม่มีโนดซ้าย หยุด Push
- Pop 35 **Process 35** Stack:NULL, 45, -77, 22, 30

- Pop 30 **Process 30** Stack:NULL, 45, -77, 22
- Pop 22 **Process 22** Stack:NULL, 45, -77
- Pop -77 เครื่องหมายเป็นลบ กลับเครื่องหมายเป็นบวก 77 ดำเนินการ **Push** ซ้ำ
เริ่ม Push 77 Stack:NULL, 45, 77 โหนด 77 มีโหนด ขวา
- Push -90 Stack:NULL, 45, 77, -90 ไม่มีโหนดซ้าย หยุด Push
- Pop -90 เครื่องหมายเป็นลบ กลับเครื่องหมายเป็นบวก 90 ดำเนินการ **Push** ซ้ำ
เริ่ม Push 88 Stack:NULL, 45, 77, 90, 88 ไม่มีโหนดซ้าย หยุด Push
- Pop 88 **Process 88** Stack:NULL, 45, 77, 90
- Pop 90 **Process 90** Stack:NULL, 45, 77
- Pop 77 **Process 77** Stack:NULL, 45
- Pop 45 **Process 45** Stack:NULL
- Pop NULL จบงาน



การทดลอง

ตัวอย่าง 7.11 การท่องตามแนวคิดของการเขียนโปรแกรม การท่องแบบโพสออร์เดอร์

```
1. public static void POSTORDER(Node root){
2.     STACK[1]=null;
3.     PTR=root;
4.     Step2();
5. }
6. public static void Step2(){
7.     while(PTR != null){ //Line 2
8.         TOP++; STACK[TOP] = PTR; mark[TOP] = 1;
9.         if(PTR.RIGHT!=null){
10.            TOP++; STACK[TOP] = PTR.RIGHT; mark[TOP]=-1;
11.        }
12.        PTR = PTR.LEFT;
```

```
13.     }//end of while loop Line 2
14.     Step3(); //call Step3
15. }
16. public static void Step3(){
17.     PTR=STACK[TOP]; indexmark= mark[TOP]; TOP--;
18.     while(indexmark>0){ //Line 10
19.         System.out.println(PTR.INFOR);
20.         PTR=STACK[TOP]; indexmark= mark[TOP]; TOP--;
21.     }
22.     if(indexmark<0){ //Line 12
23.         Step2();
24.     }
25. }
26. }
```

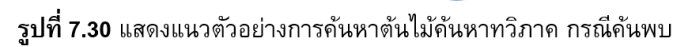
นอกจากนั้นยังสามารถเขียนเป็นแบบเวียนเกิดได้ แสดงรหัสโปรแกรมการท่องต้นไม้ค้นหาวิภาคแบบโพสต์ออร์เดอร์แบบเวียนเกิดได้ดังนี้

```
1. public static void postOrderRecursive(Node root){
2.     if(root != null){
3.         postOrderRecursive(root.LEFT);
4.         postOrderRecursive(root.RIGHT);
5.         System.out.println(root.INFOR);
6.     }
7. }
```



การค้นหาในต้นไม้ค้นหาทวิภาค

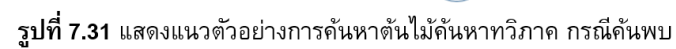
- แนวคิดการค้นหาเริ่มต้นเปรียบเทียบที่ค่าราก หากเท่ากันกับค่าราก แสดงผล หากไม่เท่าให้เทียบว่าน้อยกว่าหรือมากกว่า หากน้อยกว่าให้เทียบกับโนดด้านซ้าย หากมากกว่าให้เทียบไปทางโนดด้านขวา ดำเนินการเปรียบเทียบเช่นนี้ต่อไป จนพบข้อมูลหรือเทียบไปจนหมดโนดเชื่อมต่อ แสดงตัวอย่างการค้นหาตามแนวคิดดังรูปที่ **7.30** โดยกำหนด **ITEM** คือข้อมูลที่ต้องการค้นหา



1. กำหนด ITEM=25 (กรณีค้นพบ)

3. เปรียบเทียบไปทางซ้าย ITEM=25 เทียบกับ 22 พบว่า 25 มากกว่า 22

5. เปรียบเทียบไปทางซ้าย ITEM=25 เทียบกับ 25 พบว่า เท่ากัน รายงานผลการค้นพบ



- 
1. กำหนด ITEM=82 (กรณีค้นหาไม่พบ)
 2. เปรียบเทียบ ITEM=82 กับรากคือ 45 พบว่า 82 มากกว่า 45
 3. เปรียบเทียบไปทางขวาของราก ITEM=82 กับ 77 พบว่า 82 มากกว่า 77
 4. เปรียบเทียบไปทางขวา ITEM=82 กับ 90 พบว่า 82 น้อยกว่า 90
 5. เปรียบเทียบไปทางซ้าย ITEM=82 กับ 88 พบว่า 82 น้อยกว่า 88
 6. เปรียบเทียบไปทางซ้าย แต่ไม่มีโนดให้เทียบจึงหยุดเทียบ และรายงานผลว่าไม่พบ
จบการค้น

แนวคิดการค้นหาและอัลกอริทึมดังนี้

แนวคิดการค้นหาข้อมูลในต้นไม้ค้นหาทวิภาค โดยมีตัวแปร INFO , RIGHT , ROOT , ITEM , LOC , PAR เพื่อค้นหา ITEM ผลจะได้ตำแหน่ง LOC คือ ตำแหน่งของ ITEM และ PAR คือ ตำแหน่งของโนดแม่ของ LOC ขั้นตอนดำเนินการดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย:

1. เปรียบเทียบ ITEM กับโนดรากของต้นไม้
 - ถ้า $ITEM < \text{ค่าในโนด}$ เปรียบเทียบไปทางโนดซ้าย
 - ถ้า $ITEM > \text{ค่าในโนด}$ เปรียบเทียบไปทางโนดขวา
2. ทำซ้ำขั้นตอน 1 จนกระทั่งพบเงื่อนไขใด เงื่อนไขหนึ่งต่อไปนี้
 - ค่า ITEM เท่ากับค่าในโนดที่เทียบ รายงานผลว่าค้นพบ
 - พบต้นไม้ย่อยว่าง แสดงว่าค้นจนโนดสุดท้ายแล้ว แต่ไม่พบ ให้รายงานว่าไม่พบ

Algorithm 7.4 : FIND

Input : INFO, RIGHT, ROOT, ITEM, LOC, PAR

Output : LOC and PAR

ก่อนการใช้งานต้องพิจารณากรณีพิเศษต่อไปนี้:

- (A) $LOC = NULL$ and $PAR = NULL$ แสดงว่าต้นไม้ว่าง
- (B) $LOC \neq NULL$ and $PAR = NULL$ แสดงว่าพบ ITEM ที่ราก
- (C) $LOC = NULL$ and $PAR \neq NULL$ แสดงว่า ITEM ไม่ปรากฏในต้นไม้
1. If $ROOT = NULL$ Then //กรณีต้นไม้ว่าง
 2. $LOC := NULL$ and $PAR := NULL$
 3. Return LOC and PAR
 4. End of If
 5. If $ITEM = INFO[ROOT]$ Then //พบข้อมูลที่ราก
 6. $LOC := ROOT$ and $PAR := NULL$
 7. Return LOC and PAR



```
8. End of If
9. If ITEM < INFO[ROOT] Then //เริ่มกำหนดการค้นจากโนดถัดจากราก
10.   PTR := LEFT[ROOT] and SAVE := ROOT
11. Else:
12.   PTR := RIGHT[ROOT] and SAVE := ROOT
13. End of If
14. While PTR ≠ NULL Do
15.   If ITEM = INFO[PTR] Then //พบข้อมูล
16.     LOC := PTR and PAR := SAVE
17.     Return LOC and PAR
18.   End of If
19.   If ITEM < INFO[PTR], Then //กรณีน้อยกว่าโนดที่เทียบ
20.     SAVE := PTR and PTR := LEFT[PTR]
21.   Else //กรณีมากกว่าโนดที่เทียบ
22.     SAVE := PTR and PTR := RIGHT[PTR]
23.   End of If
24. End of While
25. LOC := NULL and PAR := SAVE
26. Return LOC and PAR //กรณีค้นแล้วไม่พบ
```

การทดลอง

ตัวอย่างที่ 7.14 จงเขียนโปรแกรมแสดงอัลกอริทึมการค้นหาคำด้วยใช้ภาษาจาวา

เฉลย : แสดงรหัสโปรแกรมส่วนการค้นหาดังนี้

```
1. //กำหนดตัวแปรสาธารณะเพื่อเรียกใช้ในเมธอดค้นหา
2.     static Node ROOT,LOC,SAVE,PAR,PTR; //กำหนดตัวแปร ROOT
3.     static int ITEM;
4.     static Node STACK[]= new Node[10];
5. public static boolean FIND(){
6.     if(ROOT==null) {LOC=null; PAR=null; return false; }
7.     if(ITEM==(int)ROOT.INFOR) {LOC=ROOT; PAR=null; return true; }
8.     if(ITEM<(int)ROOT.INFOR) {
9.         PTR=ROOT.LEFT;
10.        SAVE=ROOT;
11.    }else
12.    {
13.        PTR=ROOT.RIGHT;
14.        SAVE=ROOT;
15.    } //end if
16.    while(PTR !=null){
17.        if(ITEM==(int) PTR.INFOR) {
18.            LOC=PTR;
```



```
19.     PTR=SAVE;
20.     return true;
21. }
22. if(ITEM<(int)PTR.INFOR){
23.     SAVE=PTR;
24.     PTR=PTR.LEFT;
25. }else
26. {
27.     SAVE=PTR;
28.     PTR = PTR.RIGHT;
29. }
30. }//end while
31. LOC=null; PAR=SAVE;
32. return false;
33. } //end method
```

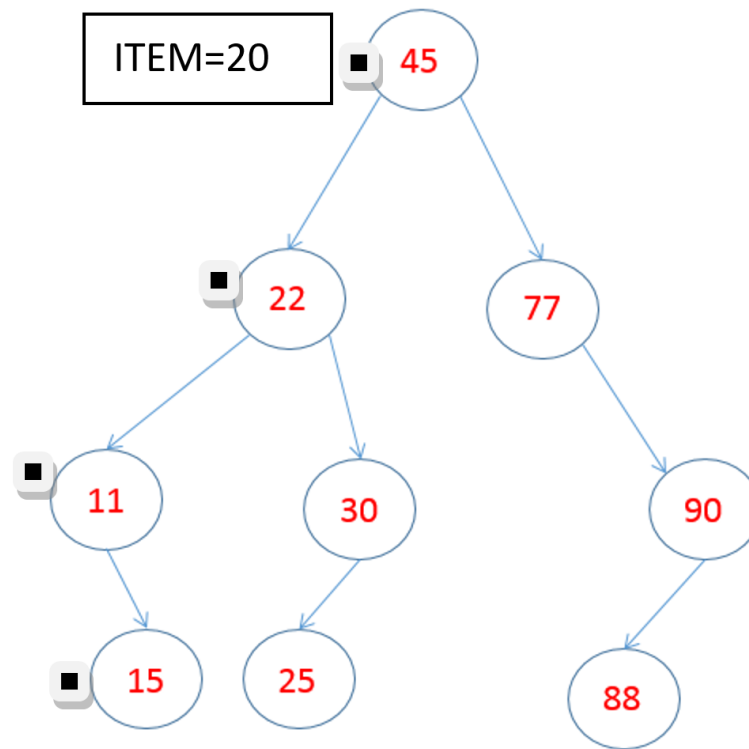


การแทรกข้อมูลลงในต้นไม้ค้นหาทวิภาค

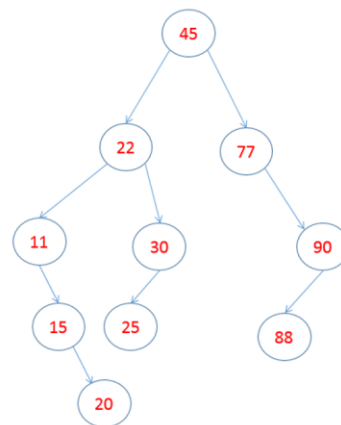
- การแทรกข้อมูลในต้นไม้ค้นหาทวิภาค เริ่มจากการค้นหาตำแหน่งของโนดที่เหมาะสมที่จะแทรก โดยใช้อัลกอริทึมการค้นหา หลังจากนั้นแทรกลงไป ในตำแหน่งที่เหมาะสมโดยถ้าค่าที่จะแทรกน้อยกว่าค่าที่โนดแม่ให้แทรก ด้านซ้ายและหากมากกว่าให้แทรกด้านขวา แสดงตัวอย่างแนวคิดดังรูปที่

7.32

แสดงการค้นหาเพื่อแทรก 20 กำหนด ITEM = 20



รูปที่ 7.32(ก) แสดงค้นหาเพื่อและแทรก 20



รูปที่ 7.32 (ข) แทรก 20 ลงไปในตำแหน่งที่เหมาะสม

จากรูปที่ 7.32 อธิบายตามแนวคิดได้ว่า

1. เปรียบเทียบ ITEM = 20 กับรากคือ 45 เนื่องจาก $20 < 45$ ทำไปทางซ้ายของ 45 ซึ่งเท่ากับ 22
 2. เปรียบเทียบ ITEM = 20 กับ 22 เนื่องจาก $20 < 22$ ทำไปทางซ้ายของ 22 คือ 11
 3. เปรียบเทียบ ITEM = 20 กับ 11 เนื่องจาก $20 > 11$ ทำไปทางขวาของ 11 คือ 15
 4. เปรียบเทียบ ITEM = 20 กับ 15 เนื่องจาก $20 > 15$ และ 15 ไม่มีโนดลูกด้านขวา
- แทรก 20 เป็นโนดลูกด้านขวาของ 15

แนวคิดการแทรกและอัลกอริทึมแสดงดังนี้

แนวคิดการแทรกโนดใหม่เข้าไปในต้นไม้ค้นหาทวิภาค โดยมีตัวแปร INFO, LEFT, RIGHT, ROOT, ITEM, LOC โดยที่โนดใหม่ ITEM จะถูกเพิ่มไปที่ตำแหน่ง LOC ขั้นตอนดำเนินการดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย :

1. กำหนด ITEM คือข้อมูลที่ต้องการค้นหา
2. (a) เปรียบเทียบ ITEM กับโน้ดรากของต้นไม้
 1. ถ้า $ITEM < \text{ค่าในโน้ด}$ เปรียบเทียบไปทางโน้ดซ้าย
 2. ถ้า $ITEM > \text{ค่าในโน้ด}$ เปรียบเทียบไปทางโน้ดขวา
3. (b) ทำซ้ำขั้นตอน (a) จนกระทั่งพบเงื่อนไขใด เงื่อนไขหนึ่งต่อไปนี้
 1. ค่า ITEM เท่ากับค่าในโน้ดที่เทียบ รายงานผลว่าค้นพบ
 2. พบต้นไม้ว่าง
4. (c) แทรก ITEM ลงในตำแหน่งต้นไม้ว่างนั้น

Algorithm 7.5: INSBST

Input : INFO, LEFT, RIGHT, ROOT, ITEM, LOC

Output : ITEM is inserted at location LOC or rejected.

1. LOC := Call FIND().
2. If LOC $\neq$ NULL Then
3. Return //มีโน้ดดังกล่าวแล้ว ไม่ต้องเพิ่ม
4. End of If
5. INFO[NEW] := ITEM //กำหนดค่าให้โน้ดใหม่
6. LOC := NEW, LEFT[NEW] := NULL and RIGHT[NEW] := NULL.
7. If PAR = NULL Then // เพิ่มโน้ดแรกที่ราก กรณีต้นไม้ว่าง
8. ROOT := NEW.
9. Else
10. If ITEM < INFO [PAR] Then
11. LEFT[PAR] := NEW //เพิ่มโน้ดซ้าย
12. Else
13. RIGHT[PAR] := NEW //เพิ่มโน้ดขวา
15. End of If
16. Return

การทดลอง

ตัวอย่างที่ 7.15 จงแสดงการเขียนโปรแกรมสำหรับอัลกอริทึมแทรกโนดใหม่ เข้าไปในต้นไม้

เฉลย: แนวทางการเขียนโปรแกรมออกแบบส่วนของโปรแกรม คือ เมธอด INSBST()

เป็นเมธอดสำหรับแทรกข้อมูลโดยการเรียกใช้ FIND() เพื่อหาตำแหน่ง LOC

```
1. //ส่วนการกำหนดเมธอดและตัวแปรสาธารณะดังที่กล่าวไปแล้ว
2. public static boolean INSBST(){
3.     FIND();
4.     if(LOC!=null)
5.         return false;
6.     else
```

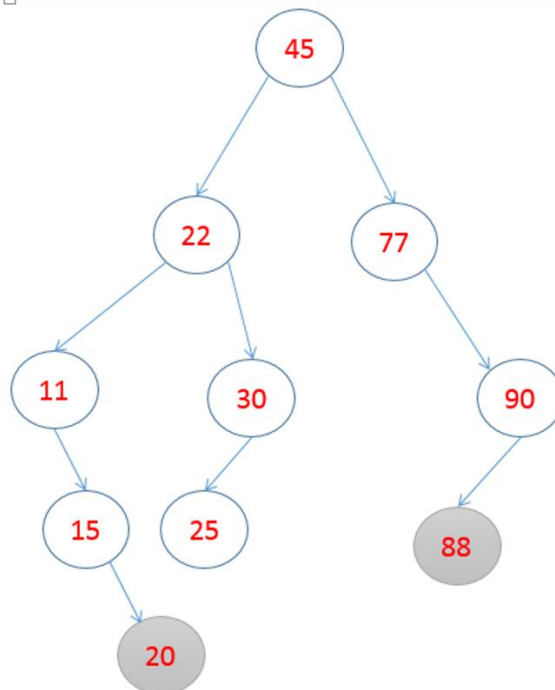
```
7.         NEW.INFOR = ITEM;
8.         if(PAR!=null){
9.             ROOT = NEW;
10.        }else
11.        if(ITEM < (int)PAR.INFOR)
12.            PAR.LEFT = NEW;
13.        else
14.            PAR.RIGHT =NEW;
15.        return true;
16.    }
```



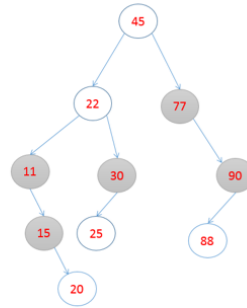
การลบข้อมูลออกจากต้นไม้ค้นหาทวิภาค

- แนวทางการลบโนดออกจากต้นไม้แบบต้นไม้ค้นหาทวิภาค ต้องคำนึงถึงเมื่อลบข้อมูลไปแล้ว จะต้องทำให้ต้นไม้ยังคงเป็นไปตามข้อกำหนดต้นไม้ค้นหาทวิภาคเสมอ กล่าวคือ ต้นไม้จะต้องยังคงใช้โนดแม่แบ่งข้อมูลสองโนดให้ข้อมูลทางซ้ายต้องน้อยกว่าข้อมูลในของโนดแม่หรือราก ขณะเดียวกันข้อมูลโนดด้านขวาต้องมากกว่าเสมอ การลบจึงมีความซับซ้อนสูง และถือว่าเป็นข้อด้อยประการหนึ่งของโครงสร้างต้นไม้ อีกทั้ง หลังจากการลบแล้วจะทำให้ต้นไม้เกิดการเสียสมดุล โดยที่ไม่คงรูปร่างเดิม โดยทั่วไปแล้ว จะเกิดการลบข้อมูลอยู่ 3 กรณี คือ
 - 1) กรณีโนดไม่มีลูกเชื่อมต่อ
 - 2) กรณีที่ลูกเพียงโนดเดียว
 - 3) กรณีมีโนดลูกสองโนด

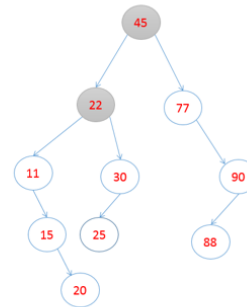
แสดงกรณีดังกล่าวดังรูปที่ 7.33



รูปที่ 7.33 (ก) แสดงกรณีไม่มีโนดลูก



รูปที่ 7.33 (ข) แสดงกรณีที่มีโนดลูกเพียงหนึ่งโนด



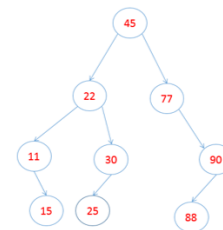
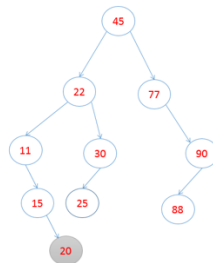
รูปที่ 7.33 (ค) แสดงกรณีที่มีโนดลูกสองโนด

แนวทางการลบโนดทั้งสามกรณี ดังนี้

หากเป็นกรณีโนดไม่มีโนดลูกสามารถลบออกไปจากต้นไม้ได้เลยทันที ทั้งนี้ จะต้องค้นหาโนดที่ต้องการลบให้พบเสียก่อน จากนั้น พิจารณาว่าเป็นกรณีการลบแบบใด โดยที่กรณีลบโนดไม่มีโนดลูกสามารถลบออกได้เลย โดยให้ค่า NULL ที่ตำแหน่งเชื่อมต่อมาของโนดแม่ที่เชื่อมมายังโนดเป้าหมาย แต่หากเป็นกรณีลบโนดที่มีโนดลูกเพียงโนดเดียว ให้ลบโนดนั้นได้เลย แล้วเอาโนดลูกเลื่อนขึ้นมาแทนโนดที่ลบไป โดยที่ให้เอาตัวชี้ของโนดแม่ชี้ไปยังโนดลูกของโนดเป้าหมาย แต่หากเป็นกรณีลบโนดที่มีโนดลูกสองโนด ให้ลบโนดนั้นได้เลย จากนั้นให้นำเอาโนดที่เป็นอินเนอร์โนดต่อไปมาแทนที่โนดนั้น โดยใช้วิธีการลบ ซึ่งจะทำให้เกิดการลบเพิ่มขึ้นในสองกรณี คือแบบกรณีไม่มีโนดลูกและมีโนดลูกเพียงโนดเดียว

แสดงแนวคิดและตัวอย่างรูปการลบ ดังนี้

กรณี ลบโนด 20 (กรณีไม่มีโนดลูก)

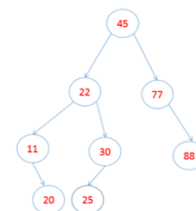
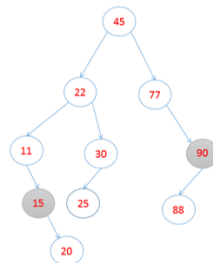


ก่อนลบ

หลังลบ

รูปที่ 7.34 แสดงกรณีการลบโนด 44 ออกจากต้นไม้ค้นหาทวิภาค

กรณี ลบโนด 15 และ 90 (กรณีมีโนดลูกเพียงโนดเดียว)

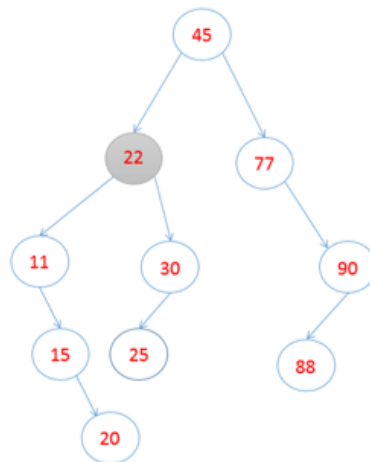


ก่อนลบ

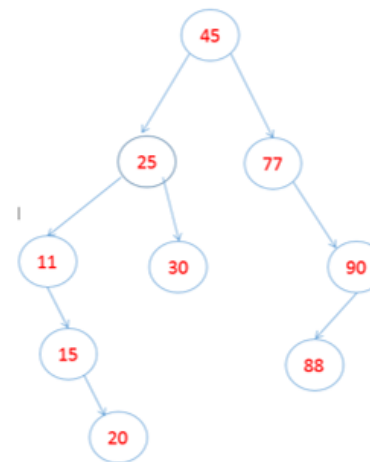
หลังลบ

รูปที่ 7.35 แสดงกรณีการลบโนด 15 และ 90 ออกจากต้นไม้ค้นหาทวิภาค

กรณี ลบ 22 (กรณีมีโนดลูกสองโนด)



ก่อนลบ



หลังลบ

รูปที่ 7.36 แสดงกรณีการลบโนด 22 ออกจากต้นไม้ค้นหาทวิภาค

คำอธิบายเพิ่มเติม

1. โหนด 22 มี 25 เป็นอินนอร์เตอร์
2. หลังจากลบโนดแล้วจะต้องเอา 25 มาแทน
3. แต่การแทนจะต้องลบ 25 ออกก่อน ซึ่งเมื่อลบ 25 จะเข้ากรณีที่ไม่มีโนดลูก

แนวคิดและอัลกอริทึมหลักแสดงดังนี้

กำหนดตัวแปร INFO, LEFT, RIGHT, ROOT, ITEM ขั้นตอนดำเนินการดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย :

- 1 กรณีลบโนดที่ไม่มีโนดลูก
 - a. สามารถลบออกได้เลย โดยให้ค่า NULL ที่ตำแหน่งเชื่อมต่อมายังโนดนั้น ในโนดแม่
- 2 กรณีลบโนดที่มีโนดลูกเพียงโนดเดียว
 - b. ให้ลบโนดนั้นได้เลย แล้ว เอาโนดลูกมาแทนโนดที่ลบ
 - c. กรณีลบโนดที่มีโนดลูกสองโนดให้ลบโนดนั้นได้เลย จากนั้นให้นำเอาโนดที่เป็นอินนอร์เตอร์โนดต่อไปมาแทนที่โนดนั้น โดยวิธีการลบ ซึ่งจะเกิดขึ้นในสองกรณี

Algorithm 7.6 : DEL

Input : INFO, LEFT, RIGHT, ROOT, ITEM

Output : ITEM is deleted from BST.

1. LOC := Call FIND().
2. If LOC = NULL Then
3. Print "ITEM not in BST"
4. Return
5. End of If
6. If RIGHT[LOC] $\neq$ NULL and LEFT[LOC] $\neq$ NULL Then
7. Call CASEB()
8. Else
9. Call CASEA()
10. End of If
11. Return

แนวคิดการลบโนดในต้นไม้ค้นหาทวิภาค โดยมีตัวแปร INFO, LEFT, RIGHT, ROOT, LOC, PAR ขั้นตอนดำเนินการดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย CASEA:

1. ถ้า LEFT[LOC] และ RIGHT[LOC] เป็น NULL กำหนด CHILD = NULL.
2. ถ้าไม่ใช่ 1 ถ้า LEFT[LOC] $\neq$ NULL กำหนด CHILD = LEFT[LOC] กรณีอื่นกำหนด CHILD = RIGHT[LOC] และจบกรณี 1
3. ถ้า PAR $\neq$ NULL และถ้า LOC = LEFT[PAR] กำหนด LEFT[PAR] = CHILD ถ้าไม่ใช่ กำหนด RIGHT[PAR] = CHILD จบกรณี 3
4. ถ้าไม่ใช่ 3 กำหนด ROOT = CHILD.
5. คืนกลับ

Algorithm 7.7: CASEA

Input : INFO, LEFT, RIGHT, ROOT, LOC, PAR

Output : ITEM is deleted from BST.

1. If LEFT[LOC] = NULL and RIGHT[LOC] = NULL, Then
2. CHILD := NULL.
3. Else
4. If LEFT[LOC] $\neq$ NULL, Then
5. CHILD := LEFT[LOC]
6. Else
7. CHILD := RIGHT[LOC].
8. End of If
9. If PAR $\neq$ NULL Then
10. If LOC = LEFT[PAR] Then:
11. LEFT[PAR] := CHILD.
12. Else.
13. RIGHT[PAR] := CHILD.
14. End of If
15. Else.
16. ROOT := CHILD.
17. End of If
18. Return.

แนวคิดและอัลกอริทึมในการลบโนดที่มีลูกสองโนด แสดงดังนี้

แนวคิดการลบโนดในต้นไม้ค้นหาทวิภาค โดยมีตัวแปร INFO, LEFT, RIGHT, ROOT, LOC, PAR ขั้นตอนดำเนินการดังนี้

แนวคิดอัลกอริทึมเชิงบรรยาย CASEB:

1. (a) กำหนด $PTR := RIGHT[LOC]$ และ $SAVE := LOC$.
(b) ในขณะที่ $LEFT[PTR] \neq NULL$:
กำหนด $SAVE := PTR$ และ $PTR := LEFT[PTR]$.
(c) กำหนด $SUC := PTR$ และ $PARSUC := SAVE$.
2. จบโดยการเรียกใช้ CASEA ().
3. แทนค่าโน้ตอินนอร์เตอร์
(a) ถ้า $PAR \neq NULL$
ถ้า $LOC = LEFT[PAR]$ กำหนด $LEFT[PAR] := SUC$ ถ้าไม่ใช่
กำหนด $RIGHT[PAR] := SUC$.
ถ้าไม่ใช่กรณี (a) กำหนด $ROOT := SUC$.
(b) กำหนด $LEFT[SUC] := LEFT[LOC]$ และ $RIGHT[SUC] := RIGHT[LOC]$
4. คืนค่า

Algorithm 7.8: CASEB

Input : INFO, LEFT, RIGHT, ROOT, LOC, PAR

Output : ITEM is deleted from BST.

1. $PTR := RIGHT[LOC]$ and $SAVE := LOC$.
2. While $LEFT[PTR] \neq NULL$ Do
3. $SAVE := PTR$ and $PTR := LEFT[PTR]$.
4. End of While
5. $SUC := PTR$ and $PARSUC := SAVE$.
6. Call CASEA ().
7. If $PAR \neq NULL$ Then
8. If $LOC = LEFT[PAR]$ Then
9. $LEFT[PAR] := SUC$.
10. Else:
11. $RIGHT[PAR] := SUC$.
12. End of If
13. Else
14. $ROOT := SUC$
15. End of If
16. $LEFT[SUC] := LEFT[LOC]$ and $RIGHT[SUC] := RIGHT[LOC]$.
17. Return.

ตัวอย่างที่ 7.16 จงเขียนโปรแกรมสำหรับอัลกอริทึมการลบโนด

```
1. static Node ROOT,LOC,SAVE,PAR,PTR, CHILD, SUC, PARSUC;
2. //กำหนดตัวแปรและเมฆอดดงที่แสดงไว้แล้ว
3. public static void DEL(){
4.     FIND(ITEM);
5.     if(LOC==null) return;
6.     if(LOC.RIGHT!=null && LOC.LEFT!=null)
7.         CASEB();
8.     else
9.         CASEA();
10. }
11. public static void CASEA(){
12.     if(LOC.LEFT==null && LOC.RIGHT==null)
13.         CHILD =null;
14.     else
15.         if(LOC.LEFT!=null)
16.             CHILD = LOC.LEFT;
17.         else
18.             CHILD = LOC.RIGHT;
19.     if(PAR!=null)
20.         if(LOC==PAR.LEFT)
21.             PAR.LEFT = CHILD;
22.         else
23.             PAR.RIGHT = CHILD;
24.     else
25.         ROOT = CHILD;
26. }
27. public static void CASEB(){
28.     PTR = LOC.RIGHT; SAVE = LOC;
29.     while(PTR.LEFT!=null){
30.         SAVE = PTR;
31.         PTR = PTR.LEFT;
32.     }
33.     SUC = PTR; PARSUC = SAVE;
34.     Node tempLOC=LOC, tempPAR=PAR; //จำไว้ชั่วคราว
35.     LOC = SUC; PAR=PARSUC;
36.     CASEA(); //ลบส่วนที่เป็นการเรียกใช้กรณีแรก A
37.     LOC = tempLOC; PAR=tempPAR;
38.     if(PAR!=null)
39.         if(LOC == PAR.LEFT)
```

```
40.         PAR.LEFT = SUC;
41.     else
42.         PAR.RIGHT = SUC;
43.     else
44.         ROOT = SUC;
45.         SUC.LEFT = LOC.LEFT;
46.         SUC.RIGHT = LOC.RIGHT;
47. }
```



Java ADT

- `Java.util.TreeSet`
- `Java.util.TreeMap`

7.8 การใช้โครงสร้างต้นไม้ด้วยภาษาจาวา

โครงสร้างข้อมูลขั้นสูงของภาษาจาวามีแพ็คเกจที่น่าสนใจคือ `java.util.TreeSet` และ `java.util.TreeMap` หากต้องการใช้งานเพียง `import` เข้ามาในโปรแกรมที่ต้องการใช้ ซึ่งมีเมธอดต่างๆ ดังนี้

1) ตัวแปรวัตถุของ `TreeSet` ดังนี้

<code>ceilingEntry(Object k)</code>	<code>Entry</code>	<code>forEach(BiConsumer bc)</code>	<code>void</code>
<code>ceilingKey(Object k)</code>	<code>Object</code>	<code>get(Object o)</code>	<code>Object</code>
<code>clear()</code>	<code>void</code>	<code>getClass()</code>	<code>Class<?></code>
<code>clone()</code>	<code>Object</code>	<code>getOrDefault(Object o, Object v)</code>	<code>Object</code>
<code>comparator()</code>	<code>Comparator</code>	<code>hashCode()</code>	<code>int</code>
<code>compute(Object k, BiFunction bf)</code>	<code>Object</code>	<code>headMap(Object k)</code>	<code>SortedMap</code>
<code>computeIfAbsent(Object k, Function fnctn)</code>	<code>Object</code>	<code>headMap(Object k, boolean bin)</code>	<code>NavigableMap</code>
<code>computeIfPresent(Object k, BiFunction bf)</code>	<code>Object</code>	<code>higherEntry(Object k)</code>	<code>Entry</code>
<code>containsKey(Object o)</code>	<code>boolean</code>	<code>higherKey(Object k)</code>	<code>Object</code>
<code>containsValue(Object o)</code>	<code>boolean</code>	<code>isEmpty()</code>	<code>boolean</code>
<code>descendingKeySet()</code>	<code>NavigableSet</code>	<code>keySet()</code>	<code>Set</code>
<code>descendingMap()</code>	<code>NavigableMap</code>	<code>lastEntry()</code>	<code>Entry</code>
<code>entrySet()</code>	<code>Set</code>	<code>lastKey()</code>	<code>Object</code>
<code>equals(Object o)</code>	<code>boolean</code>	<code>lowerEntry(Object k)</code>	<code>Entry</code>
<code>firstEntry()</code>	<code>Entry</code>	<code>lowerKey(Object k)</code>	<code>Object</code>
<code>firstKey()</code>	<code>Object</code>	<code>merge(Object k, Object v, BiFunction bf)</code>	<code>Object</code>
<code>floorEntry(Object k)</code>	<code>Entry</code>	<code>navigableKeySet()</code>	<code>NavigableSet</code>
	<code>pollFirstEntry()</code>	<code>Entry</code>	
	<code>pollLastEntry()</code>	<code>Entry</code>	
	<code>put(Object k, Object v)</code>	<code>Object</code>	
	<code>putAll(Map map)</code>	<code>void</code>	
	<code>putIfAbsent(Object k, Object v)</code>	<code>Object</code>	
	<code>remove(Object o)</code>	<code>Object</code>	
	<code>remove(Object o, Object ol)</code>	<code>boolean</code>	
	<code>replace(Object k, Object v)</code>	<code>Object</code>	
	<code>replace(Object k, Object v, Object vl)</code>	<code>boolean</code>	
	<code>replaceAll(BiFunction bf)</code>	<code>void</code>	
	<code>size()</code>	<code>int</code>	
	<code>subMap(Object k, Object kl)</code>	<code>SortedMap</code>	
	<code>subMap(Object k, boolean bin, Object kl, boolean binl)</code>	<code>NavigableMap</code>	
	<code>tailMap(Object k)</code>	<code>SortedMap</code>	
	<code>tailMap(Object k, boolean bin)</code>	<code>NavigableMap</code>	
	<code>toString()</code>	<code>String</code>	
	<code>values()</code>	<code>Collection</code>	

รูปที่ 7.28 แสดงเมธอดของ `java.util.TreeSet`

<code>add(Object e)</code>	boolean	<code>headSet(Object e, boolean bln)</code>	NavigableSet
<code>addAll(Collection clctn)</code>	boolean	<code>higher(Object e)</code>	Object
<code>ceiling(Object e)</code>	Object	<code>isEmpty()</code>	boolean
<code>clear()</code>	void	<code>iterator()</code>	Iterator
<code>clone()</code>	Object	<code>last()</code>	Object
<code>comparator()</code>	Comparator	<code>lower(Object e)</code>	Object
<code>contains(Object o)</code>	boolean	<code>notify()</code>	void
<code>containsAll(Collection clctn)</code>	boolean	<code>notifyAll()</code>	void
<code>descendingIterator()</code>	Iterator	<code>parallelStream()</code>	Stream
<code>descendingSet()</code>	NavigableSet	<code>pollFirst()</code>	Object
<code>equals(Object o)</code>	boolean	<code>pollLast()</code>	Object
<code>first()</code>	Object	<code>remove(Object o)</code>	boolean
<code>floor(Object e)</code>	Object	<code>removeAll(Collection clctn)</code>	boolean
<code>forEach(Consumer cnsmr)</code>	void	<code>removeIf(Predicate prdct)</code>	boolean
<code>getClass()</code>	Class<?>	<code>retainAll(Collection clctn)</code>	boolean
<code>hashCode()</code>	int	<code>size()</code>	int
<code>headSet(Object e)</code>	SortedSet	<code>splitterator()</code>	Splitterator
		<code>remove(Object o)</code>	boolean
		<code>removeAll(Collection clctn)</code>	boolean
		<code>removeIf(Predicate prdct)</code>	boolean
		<code>retainAll(Collection clctn)</code>	boolean
		<code>size()</code>	int
		<code>splitterator()</code>	Splitterator
		<code>stream()</code>	Stream
		<code>subSet(Object e, Object e1)</code>	SortedSet
		<code>subSet(Object e, boolean bln, Object e1, boolean bln1)</code>	NavigableSet
		<code>tailSet(Object e)</code>	SortedSet
		<code>tailSet(Object e, boolean bln)</code>	NavigableSet
		<code>toArray()</code>	Object[]
		<code>toArray(Object[] ts)</code>	Object[]
		<code>toString()</code>	String
		<code>wait()</code>	void
		<code>wait(long l)</code>	void
		<code>wait(long l, int i)</code>	void

รูปที่ 7.29 แสดงเมทอดของ java.util.TreeMap



- import java.util.TreeMap;
- /**
- *
- * @author ComSCiv3400
- */
- public class useTreeMap {
- public static void main(String args[]){
- TreeMap a = new TreeMap();
- a.put(1, "aaa");
- a.put(2, "bbb");
- a.put(3, "ccc");
- a.put(4, "ddd");
- a.put(5, "eee");
- System.out.println(a);
- System.out.println(a.values());
- System.out.println(a.get(4));
- }
- }



- `import java.util.TreeSet;`
- `/**`
- `*`
- `* @author ComSClv3400`
- `*/`
- `public class useTreeSet {`
- `public static void main(String args[]){`
- `TreeSet a = new TreeSet();`
- `a.add(14);`
- `a.add(30);`
- `a.add(50);`
- `a.add(12);`
- `a.add(3);`
- `System.out.println(a);`
- `}`
- `}`

บูรณาการโครงสร้างต้นไม้ค้นหาทวิภาคกับการเขียนโปรแกรมเชิงวัตถุ

- การบูรณาการโดยการนำอัลกอริทึมทั้งหมดที่ได้นำเสนอไป นำมาสร้างเป็นคลาสสำหรับนำไปใช้ในโปรแกรม การออกแบบกำหนดแพ็คเกจ **integrative** ซึ่งภายในประกอบคลาส **Node** เป็นคลาสโนดข้อมูล **RUTree** เป็นคลาสหลักเชิงวัตถุ และ **RUTreeApp** เป็นตัวอย่างการใช้งานคลาส **RUTree** แสดงดังรูปที่ 7.32



รูปที่ 7.30 แสดงคลาสบูรณาการโครงสร้างต้นไม้ค้นหาทวิภาค

รหัสโปรแกรมแต่ละคลาส ดังนี้

- โปรแกรม Node.java

1. package integrativeTree;
2. public class Node {
3. Object INFOR;
4. Node LEFT=null;
5. Node RIGHT=null;
6. }

- โปรแกรม RUTree.java ออกแบบให้มีเมธอดที่สำคัญในการเรียกใช้งานเป็นอัลกอริทึมที่ได้นำเสนอไปทั้งหมด แสดงรูปของเมธอดดังรูปต่อไปนี้

CHILD	Node	DEL(int ITEM)	void
LOC	Node	FIND(int ITEM)	boolean
PAR	Node	INORDER(Node root)	void
PARSUC	Node	INSBST(int ITEM)	boolean
PTR	Node	POSTORDER(Node root)	void
ROOT	Node	PREORDER(Node root)	void
SAVE	Node	Step2()	void
STACK	Node[]	Step2p()	void
SUC	Node	Step3p()	void
TOP	int	equals(Object o)	boolean
indexmark	int	getClass()	Class<?>
mark	int[]	hashCode()	int
CASEA()	void	inOrderRecursive(Node root)	void
CASEB()	void	notify()	void
DEL(int ITEM)	void	notifyAll()	void
FIND(int ITEM)	boolean	postOrderRecursive(Node root)	void
INORDER(Node root)	void	preOrderRecursive(Node root)	void

รูปที่ 7.31 แสดงตัวแปรและเมธอดของคลาส RUTree

รหัสโปรแกรมดังนี้

```

1. package integrativeTree;
2. public class RUTree {
3.     Node ROOT; //กำหนดตัวแปร ROOT
4.     Node STACK[] = new Node[100];
5.     int TOP=1;
6.     Node PTR;
7.     int mark[] = new int[100];
8.     int indexmark=0;
9.     Node LOC,SAVE,PAR;
10.    Node CHILD, SUC, PARSUC;
11.    //-----
12.    public void PREORDER(Node root){
13.        //รหัสโปรแกรมดั่งที่นำเสนอไว้แล้ว
14.    }
15.    //-----
16.    public void INORDER(Node root){
17.        //รหัสโปรแกรมดั่งที่นำเสนอไว้แล้ว
18.    //-----
19.    public void POSTORDER(Node root){
20.        //รหัสโปรแกรมดั่งที่นำเสนอไว้แล้ว
21.    }
22.    //-----
23.    public boolean FIND(int ITEM){
24.        //รหัสโปรแกรมดั่งที่นำเสนอไว้แล้ว
25.    } //end method
26.    //-----
27.    public boolean INSBST(int ITEM){

```

```

28. //รหัสโปรแกรมที่ตั้งที่นำเสนอไว้แล้ว
29. }
30. //-----
31. public void DEL(int ITEM){
32. //รหัสโปรแกรมที่ตั้งที่นำเสนอไว้แล้ว
33. }
34. //-----
35. public void preOrderRecursive(Node root){
36. //รหัสโปรแกรมที่ตั้งที่นำเสนอไว้แล้ว
37. }
38. //-----
39. public void inOrderRecursive(Node root){
40. //รหัสโปรแกรมที่ตั้งที่นำเสนอไว้แล้ว
41. }
42. //-----
43. public void postOrderRecursive(Node root){
44. //รหัสโปรแกรมที่ตั้งที่นำเสนอไว้แล้ว
45. }

```

- โปรแกรม RUTreeApp เป็นโปรแกรมที่สร้างวัตถุของ RUTree ใช้งาน รหัสโปรแกรมดังนี้

```

1. package integrativeTree;
2. public class RUTreeApp {
3.     public static void main(String args[]){
4.         RUTree mytree = new RUTree();
5.         mytree.INSBST(50);
6.         mytree.INSBST(40);
7.         mytree.INSBST(60);
8.         mytree.INSBST(30);
9.         mytree.INSBST(70);
10.        mytree.INSBST(20);
11.        mytree.INSBST(80);
12.        mytree.INSBST(10);
13.        mytree.INSBST(100);
14.        System.out.append("--- Traversing non-Recursive ---\n");
15.        mytree.PREORDER(mytree.ROOT);
16.        System.out.println();
17.        mytree.INORDER(mytree.ROOT);
18.        System.out.println();
19.        mytree.POSTORDER(mytree.ROOT);

```

```

20.        System.out.println();
21.        System.out.append("--- Traversing Recursive ---\n");
22.        mytree.preOrderRecursive(mytree.ROOT);
23.        System.out.println();
24.        mytree.inOrderRecursive(mytree.ROOT);
25.        System.out.println();
26.        mytree.postOrderRecursive(mytree.ROOT);
27.        System.out.println();
28.        System.out.append("--- Searching 70 and 120 ---\n");
29.        if(mytree.FIND(70))
30.            System.out.println("Found : 70");
31.        else
32.            System.out.println("Not Found : 70");
33.        if(mytree.FIND(120))
34.            System.out.println("Found : 120");
35.        else
36.            System.out.println("Not Found : 120");
37.        mytree.DEL(70);
38.        System.out.append("--- After Deleting 70 from Tree ---\n");
39.        mytree.PREORDER(mytree.ROOT);
40.    }
41. }

```

ผลการรันโปรแกรม :

```

--- Traversing non-Recursive ---
50 40 30 20 10 60 70 80 100
10 20 30 40 50 60 70 80 100
10 20 30 40 100 80 70 60 50
--- Traversing Recursive ---
50 40 30 20 10 60 70 80 100
10 20 30 40 50 60 70 80 100
10 20 30 40 100 80 70 60 50
--- Searching 70 and 120 ---
Found : 70
Not Found : 120
Data LOC:70
Data PAR:60
Del CASE A OK.
--- After Deleting 70 from Tree ---
50 40 30 20 10 60 80 100

```

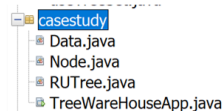
7.9.2 กรณีศึกษาโครงสร้างต้นไม้ค้นหาวิภาค

กรณีศึกษาการเขียนโปรแกรมสำหรับเก็บข้อมูลสินค้าในคลัง มีความต้องการดังนี้

1. สร้างการเก็บข้อมูลสินค้าด้วยโครงสร้างต้นไม้ที่พัฒนาขึ้นในสถานการณ์การบูรณาการก่อนหน้านี้
 2. ให้สามารถ เพิ่ม ลบ และ ค้นหา พร้อมกันนั้นสามารถแสดงการทอข้อมูลต่างๆ โดยแสดงในส่วนของการตาราง
- ออกแบบหน้าจอดังนี้

รูปที่ 7.32 แสดงหน้าจอของการออกแบบโปรแกรมระบบคลังสินค้า

การออกแบบโปรแกรม กำหนดคลาส Data เป็นคลาสข้อมูลที่ใช้บรรจุลงในวัตถุของ Node รายละเอียดของ Node.java และ RUTree.java เป็นดังที่แสดงไว้ก่อนหน้านี้ โดยมีโปรแกรม TreeWareHouseApp.java เป็นโปรแกรมหลักที่เป็นแบบกราฟิกอินเทอร์เฟซ แสดงรูปที่บรรจุโปรแกรมต่างๆ ดังนี้



รูปที่ 7.33 แสดงการออกแบบโปรแกรมในระบบจัดเก็บสินค้า

ส่วนของคลาสที่ใช้สร้างวัตถุข้อมูล Data.java ดังนี้

```
1. package casestudy;
2. public class Data {
3.     int Id;
4.     int Type;
5.     String Name;
6.     int Picecount;
7.     float PriceperPice;
8. }
```

คลาส TreeWareHouseApp ซึ่งเป็นโปรแกรมหลักมีรายละเอียดในส่วนที่สำคัญ ดังนี้

- ส่วนหัวโปรแกรม

```
1. package casestudy;
2. public class TreeWareHouseApp extends javax.swing.JFrame {
3.     RUTree mytree = new RUTree();
4.     int row = -1;
```

- ส่วนเมธอดเพิ่มเติม

```
1. public void clearTable(){
2.     for(int i=0;i<100;i++){
3.         jTable1.setValueAt("", row, 0);
4.         jTable1.setValueAt("", row, 1);
5.         jTable1.setValueAt("", row, 2);
6.         jTable1.setValueAt("", row, 3);
7.         jTable1.setValueAt("", row, 4);
8.     }
9. }
```

- ส่วนของปุ่มต่างๆ

1. ปุ่มเพิ่ม

```
1. private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
2.     Data data = new Data();
3.     data.Id = Integer.parseInt(jTextField1.getText());
4.     data.Name = jTextField2.getText();
5.     data.Type = jComboBox1.getSelectedIndex();
6.     data.Picecount = Integer.parseInt(jTextField3.getText());
7.     data.PriceperPice = Float.parseFloat(jTextField4.getText());
8.     if(mytree.INSBST(data)){
9.         row++;
10.        clearTable();
11.        jTable1.setValueAt(data.Id, row, 0);
```

ประยุกต์วิจัย

- งานวิจัยเกี่ยวกับโครงสร้างต้นไม้มีจำนวนมาก เนื่องจากโครงสร้างนี้เป็นโครงสร้างที่มีหลายรูปแบบ แต่เมื่อพิจารณางานวิจัยที่เกี่ยวข้องกับต้นไม้ค้นหาทวิภาค จากการค้นหาบทความงานวิจัยที่มีการตีพิมพ์อยู่บนฐานข้อมูลวิจัยด้านดิจิทัลในปัจจุบัน ด้วยการค้นหาจากคำสำคัญ **“binary tree”** (ข้อมูล ณ วันที่ 26 กรกฎาคม 2563) แสดงดังนี้
- ACM จำนวน 188,006 บทความ
- IEEE จำนวน 7,036 บทความ
- CiteSeerX จำนวน 4,653 บทความ
- Sci-Direct จำนวน 101,584 บทความ

แบบฝึกหัด

- แบบฝึกหัดทฤษฎี
- 1. จงอธิบายความหมายของต้นไม้ ต้นไม้ทวิภาค
- 2. จงบอกส่วนประกอบของต้นไม้
- 3. จงอธิบายและยกตัวอย่างข้อแตกต่างระหว่างต้นไม้ ต้นไม้ทวิภาค และต้นไม้ค้นหาทวิภาค
- 4. จงอธิบายและยกตัวอย่างประเภทของต้นไม้ทวิภาค
- 5. จงยกตัวอย่างและแสดงการท่องต้นไม้แบบพรีออร์เดอร์ อินออร์เดอร์ และโพสต์ออร์เดอร์
- 6. จงแสดงวิธีการค้นหาต้นไม้ ต้นไม้ทวิภาคที่มีลักษณะดังต่อไปนี้

- เมื่อคัน 3 , 5 , และ 40 ตามลำดับ
- 7. จากข้อ 6 จงอธิบายวิธีการแทรกข้อมูล 20 และ 125
- 8. จากข้อ 6 จงอธิบายการลบโนด 3 , 10 และ 13 ออกจากต้นไม้ดังกล่าว
- 9. จากข้อ 6 หากทำเป็นต้นไม้แบบกึ่งกัน (Balance) จะได้รูปใหม่เช่นไร จงอธิบาย
- 10. จงบอกข้อดี ข้อเสีย ของการใช้งานโครงสร้างต้นไม้ทวิภาค
- 11. เหตุปัจจัยใดบ้างที่จะทำให้ทราบว่า โครงสร้างต้นไม้ที่มีความสมดุล
- 12. โครงสร้างต้นไม้ค้นหาทวิภาคมีข้อดีและข้อเสียอะไร จงอธิบายและยกตัวอย่าง
- 13. เมื่อมีโครงสร้างต้นไม้ค้นหาทวิภาค 2 โครงสร้าง จะสามารถตรวจสอบได้อย่างไรว่า โครงสร้างต้นไม้ใดเป็น ต้นไม้ย่อยของอีกโครงสร้าง
- 14. จงอธิบายแนวคิด และอัลกอริทึมสำหรับตรวจสอบว่า โหนดที่อยู่ในโครงสร้างอยู่ในระดับเดียวกันหรือไม่
- 15. จงอธิบายแนวคิด และอัลกอริทึมสำหรับค้นหาความลึกของโครงสร้างต้นไม้ทวิภาค
- 16. หากโครงสร้างต้นไม้ทวิภาคที่กำหนดให้เป็นสลับกันกับคุณสมบัติเดิม กล่าวคือ โหนดด้านซ้ายมีค่ามากกว่าค่า โหนดราก และโนดขวามีค่าน้อยกว่าค่าโนดราก จงอธิบายแนวคิด และอัลกอริทึมสำหรับการกู้คืนโครงสร้างดังกล่าว

- 17. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการค้นหาต้นไม้ย่อยที่ใหญ่ที่สุดในโครงสร้างต้นไม้ทวิภาค
- 18. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการเทียบว่าโครงสร้างต้นไม้ทวิภาคที่กำหนดให้เหมือนกันหรือไม่
- 19. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการตรวจสอบว่าต้นไม้ทวิภาคที่กำหนดให้เป็นต้นไม้ทวิภาคที่สมบูรณ์หรือไม่
- 20. จงอธิบายแนวคิด และอัลกอริทึมสำหรับตรวจสอบว่าในดสองโนดเป็นลูกพี่ลูกน้องของกันและกันหรือไม่
- 21. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการแปลงโครงสร้างต้นไม้ทวิภาคใดๆ ให้เป็นโนดแบบแสดงในกระจก
- 22. กำหนดให้โนดในโครงสร้างต้นไม้ทวิภาคใดๆ เก็บข้อมูลตัวเลขจำนวนเต็ม จงอธิบายแนวคิด และอัลกอริทึมสำหรับคำนวณผลรวมแนวตั้งในต้นไม้ทวิภาคดังกล่าว
- 23. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการค้นหาความกว้างสูงสุดของต้นไม้ทวิภาคที่กำหนด
- 24. จงอธิบายแนวคิด และอัลกอริทึมสำหรับแปลงโครงสร้างต้นไม้ทวิภาคปกติ ให้เป็นโครงสร้างต้นไม้ทวิภาคที่โนดซ้ายมีค่ามากกว่าค่าโนดขวา และโนดขวามีค่าน้อยกว่าโนดขวา
- 25. จงอธิบายว่าโครงสร้างการค้นหาแบบทวิภาคทำงานอย่างไร
- 26. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการนับจำนวนโนดในโครงสร้างต้นไม้ทวิภาค
- 27. จงอธิบายแนวคิดสำหรับการสร้างโครงสร้างต้นไม้ทวิภาคโดยอาศัยโครงสร้างข้อมูลแถวลำดับ
- 29. จงอธิบายข้อแตกต่างระหว่างโครงสร้างต้นไม้ทวิภาค และโครงสร้างต้นไม้ค้นหาทวิภาค
- 30. จงอธิบายแนวคิด และอัลกอริทึมสำหรับนับต้นไม้ย่อยทั้งหมดที่มีอยู่ในโครงสร้างต้นไม้ทวิภาค
- 31. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการค้นหาความลึกที่สุดของต้นไม้ทวิภาค
- 32. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการค้นหาความลึกครั้งแรกสุดของต้นไม้ทวิภาค
- 33. จงเปรียบเทียบการท่องโครงสร้างต้นไม้ทวิภาคแบบอินออรัโดรกับการค้นหาข้อมูล ในโครงสร้างว่ามีความเหมือนหรือแตกต่างกันอย่างไร
- 34. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการเติมโนดข้างเคียงที่เหมาะสมสำหรับโนดทั้งหมดที่มีอยู่ในโครงสร้างต้นไม้ทวิภาคให้เป็นต้นไม้ที่สมบูรณ์
- 35. จงอธิบายว่า สามารถแปลงโครงสร้างต้นไม้ทวิภาคที่กำหนดให้เป็นรายการโยงแบบสองทางได้หรือไม่ เพราะเหตุใด พร้อมทั้งยกตัวอย่าง
- 36. จงอธิบายแนวคิด และอัลกอริทึมสำหรับการนับโนดปลาย (Leaf node) ในต้นไม้ทวิภาค
- 37. จงอธิบายแนวคิด และอัลกอริทึมสำหรับหาค่ามากที่สุดที่ใหญ่ที่สุดในแต่ละระดับของต้นไม้ทวิภาค
- 38. จงอธิบายว่าต้นไม้ทวิภาคมีข้อดีกว่าโครงสร้าง แถวคอย กองซ้อน แถวคอย และรายการโยงอย่างไร
- 39. จงอธิบายแนวคิด และอัลกอริทึมสำหรับหาค่ามัธยฐานของข้อมูลที่บรรจุอยู่ในโครงสร้างต้นไม้ทวิภาคโดยใช้ความซับซ้อนด้านเวลาเวลา $O(n)$ และใช้เนื้อที่ $O(1)$
- 40. จงอธิบายแนวคิด และอัลกอริทึมสำหรับลบโนดปลายออกจากต้นไม้ค้นหาทวิภาค

- **แบบฝึกหัดปฏิบัติ**

- 1. กำหนดให้ในประกอบด้วย 15, 20, 60, 50, 5 จงวาดภาพต้นไม้ดังกล่าวโดย 50 เป็นราก จากนั้นเขียนโปรแกรมเพื่อสร้างโนดทั้งหมด
- 2. จากข้อ 1 จงเขียนโปรแกรมท่องต้นไม้ดังกล่าวด้วยอัลกอริทึมแบบพรีออร์เดอร์
- 3. จากข้อ 1 จงเขียนโปรแกรมท่องต้นไม้ดังกล่าวด้วยอัลกอริทึมแบบอินออร์เดอร์
- 4. จากข้อ 1 จงเขียนโปรแกรมท่องต้นไม้ดังกล่าวด้วยอัลกอริทึมแบบโพสต์ออร์เดอร์
- 5. กำหนดให้รากคือ 60 จงเขียนโปรแกรมสำหรับเพิ่มข้อมูลเข้าไปต้นไม้วีภาคดังกล่าว
- 6. จงเขียนโปรแกรมเพื่อค้นหาข้อมูลในต้นไม้ ในข้อ 1
- 7. จงเขียนโปรแกรมลบข้อมูล โดยระบุโนดที่ต้องการลบได้
- 8. จงเขียนโปรแกรมเพื่อประยุกต์ใช้ โดยเก็บ ชื่อ สกุล ที่อยู่ และเงินเดือนของพนักงานโดยใช้โครงสร้างต้นไม้
- 9. จากข้อ 8 จงเขียนโปรแกรมให้สามารถ เพิ่ม ลบ แก้ไข ข้อมูลของต้นไม้ดังกล่าว
- 10. จงเขียนโปรแกรมสร้างต้นไม้ในข้อ 1 และข้อ 8 โดยใช้โครงสร้างอาร์เรย์ในการเก็บต้นไม้ดังกล่าว
- 11. จงเขียนโปรแกรมแสดงโนดด้านซ้ายของต้นไม้วิภาค
- 12. จงเขียนโปรแกรมตรวจสอบว่าต้นไม้วิภาคสมดุลหรือไม่ หากสมดุลคืนค่า จริง หรือเป็นเท็จเป็นอย่างอื่น
- 13. จงเขียนโปรแกรมนับจำนวนโนดปลายทั้งหมดที่มีอยู่โครงสร้างต้นไม้วิภาค
- 14. จงเขียนโปรแกรมหาความลึกของต้นไม้วิภาคที่กำหนดให้
- 15. จงเขียนโปรแกรมเพื่อแสดงการท่องต้นไม้วิภาคโดยใช้ทั้งการเวียนเกิดและการใช้การวนซ้ำ
- 16. หากโนดในต้นไม้วิภาคมีการสลับค่ากันระหว่างโนดซ้ายและขวาทุกๆ โหนด ให้เขียนโปรแกรมกู้คืนให้กลับมาเป็นต้นไม้ค้นหาวิภาค (BST)
- 17. จงเขียนโปรแกรมแสดงเส้นทางไปยังทุกๆ โหนดปลาย (Path-to-Leaf) ในโครงสร้างต้นไม้วิภาค

- 18. จงเขียนโปรแกรมลบโนดที่กำหนดออกจากโครงสร้างต้นไม้ทวิภาค
- 19. จงเขียนโปรแกรมพิมพ์โนดในต้นไม้ทวิภาคตามลำดับแนวตั้ง
- 20. จงเขียนโปรแกรมพิมพ์โนดข้างเคียง (พี่น้อง) ของโนดที่กำหนดให้ในต้นไม้ทวิภาค
- 21. เมื่อข้อมูลที่บรรจุอยู่ในโนดของต้นไม้ทวิภาคเป็นตัวเลขจำนวนเต็ม จงเขียนโปรแกรมหาผลรวมในต้นไม้ทวิภาคดังกล่าว
- 22. จงเขียนโปรแกรมเพื่อค้นหาความกว้างสูงสุดของต้นไม้ทวิภาคที่กำหนด
- 23. จงเขียนโปรแกรมพิมพ์มุมมองขวาของต้นไม้ทวิภาคที่กำหนด
- 24. จงเขียนโปรแกรมนับจำนวนโนดที่บรรจุอยู่ในต้นไม้ทวิภาคที่กำหนด
- 25. จงเขียนโปรแกรมแปลงต้นไม้ทวิภาคที่กำหนด ให้ใช้รายการที่เชื่อมโยงสองทาง
- 26. จงเขียนโปรแกรมแปลงต้นไม้ทวิภาคที่กำหนด ให้ใช้โครงสร้างแถวลำดับ
- 27. จงเขียนโปรแกรมเชื่อมต่อโนดที่อยู่ในระดับเดียวกันในโครงสร้างต้นไม้ทวิภาค
- 28. เมื่อมีโครงสร้างต้นไม้ทวิภาคใดๆ อยู่แล้ว หลังจากนั้นกำหนดค่าคีย์ซึ่งเป็นโนดที่อยู่ในโครงสร้างต้นไม้ทวิภาคดังกล่าว ให้เขียนโปรแกรมเพื่อสร้างโครงสร้างต้นไม้ทวิภาคใหม่โดยใช้ค่าคีย์ดังกล่าวเป็นค่าราก
- 29. จงเขียนโปรแกรมค้นหาค่าน้อยที่สุด และมากที่สุด ในโครงสร้างต้นไม้ทวิภาค
- 30. จงเขียนโปรแกรมลบข้อมูลเป็นช่วงในโครงสร้างต้นไม้ทวิภาค โดยให้ผู้ใช้กำหนดช่วงข้อมูลที่จะลบ
- 31. จงเขียนโปรแกรมนับจำนวนต้นไม้ย่อยที่มีอยู่ในโครงสร้างต้นไม้ทวิภาคที่กำหนด
- 33. จงเขียนโปรแกรมค้นหาขนาดของต้นไม้ค้นหาทวิภาคที่ใหญ่ที่สุดในโครงสร้างต้นไม้ทวิภาคใดๆ
- 34. เมื่อกำหนด โหนด 2 โหนด ในโครงสร้างต้นไม้ทวิภาค จงเขียนโปรแกรมตรวจสอบว่าสองโนดดังกล่าวเป็นข้างเคียงกัน (โนดพี่น้อง) หรือไม่
- 35. จงเขียนโปรแกรมพิมพ์มุมมองด้านบนและด้านล่างของโครงสร้างต้นไม้ทวิภาคใดๆ
- 36. จงเขียนโปรแกรมเติมโนดข้างเคียงที่หายไปจากโครงสร้างต้นไม้ทวิภาคให้สมบูรณ์

สรุป

- ต้นไม้เป็นโครงสร้างข้อมูลชนิดที่ไม่ต่อเนื่อง มีส่วนประกอบคือโนดและส่วนการเชื่อมโยงต่อไปยังโนดอื่นๆ ในมุมมองบนล่าง ต้นไม้มีหลายประเภท สนับสนุนการสำหรับการเข้าถึงข้อมูลรวดเร็ว ต้นไม้ค้นหาทวิภาคเป็นโครงสร้างต้นไม้ที่มีจำนวนในลิสไม่เกินสองโนด โดยมีตัวดำเนินการ คือ การท่องมี 3 แบบ คือ ฟรีออร์เดอร์ (ราก , ซ้าย , ขวา) อินออร์เดอร์ (ซ้าย , ราก , ขวา) และ โฟสออร์เดอร์ (ซ้าย , ขวา , ราก) การค้นหา เข้าที่รากหากตัวที่ต้องการหา น้อยกว่ารากจะไปค้นต่อทางด้านซ้าย หากมากกว่าจะไปค้นด้านขวาของค่ารากดำเนินการ การแทรกต้องค้นหาตำแหน่งของการแทรกด้วยอัลกอริทึม การแทรกดำเนินการกับส่วนที่เป็น **NULL** ของโนดเชื่อมต่อ การลบข้อมูลมีเงื่อนไขคือ ลบโดยมีโนดลูกเดียวสามารถลบได้ทันที ลบโนดที่มีสองโนดจะทำการลบและเลื่อนโนดที่เป็นอินออร์เดอร์เข้ามาแทนที่

References

- Seymour Lipschutz:แปลโดย อุดม จินประดับ และ ดร.สมคิด เรืองชนะสกุลไทย. ทฤษฎีและตัวอย่างโจทย์ โครงสร้างข้อมูล---**Theory and problem of Data Structures**. กรุงเทพฯ : แมคกรอ-ฮิล อินเทอร์เน็ต เนชั่นแนล เอ็นเตอร์ไพรส์, อิงค์, (2540).
- เซาวลิต ชันคำ. (2554). การเขียนโปรแกรมภาษาคอมพิวเตอร์. ฉะเชิงเทรา : มหาวิทยาลัยราชภัฏราชนครินทร์.
- เซาวลิต ชันคำ. (2564). โครงสร้างข้อมูลและอัลกอริทึม. กรุงเทพฯ : มหาวิทยาลัยรามคำแหง.